Building Multimodal Agents With NVIDIA Nemotron and RAPIDS - Part 2

Antonio Rueda-Toicen

Senior Developer Relations Manager

Vaibhav Sabhahit

Senior Solutions Architect

Agenda

- Introduction to Bertopic and Concept
- Multimodal embeddings with CLIP and CRADIO-v4
- Dimensionality Reduction with Principal Component Analysis and UMAP
- Clustering with K-means and HDBSCAN
- cuML acceleration with RAPIDS

Notebook

- Bertopic on Images (Google Colab)

Setup

- Configuring your NVIDIA_API_KEY to use models from build.nvidia.com
- Setting up GPUs on Kaggle Notebooks
- Setting up a HuggingFace token
- Setting Access to Kaggle Jupyter Server on VS Code

Core question in topic modeling

"What is this data about?"

Subjective, for both humans and LLMs

Topic Modeling

45 min

Extract insights from massive text datasets using cuML's GPU-accelerated BERTopic

bertopic | data science | machine learning | nlp | cuml

Overview Instructions

Basic idea

Topic modeling helps you discover hidden themes in large document collections—but traditional methods crawl when datasets grow to millions of records. This playbook shows how to process **40 million Amazon product reviews in minutes** using GPU-accelerated BERTopic.

BERTopic combines transformer embeddings with clustering to extract human-readable topics from text. By swapping CPU-based UMAP and HDBSCAN with GPU-accelerated versions from **RAPIDS cuML**, you get the same results dramatically faster—no code changes required.

- **Drop-in GPU acceleration**: Load `cuml.accel` and your existing UMAP/HDBSCAN code runs on GPU automatically
- **Scale to millions**: Process datasets that would take hours on CPU in minutes on GPU
- **Interactive visualizations**: Explore topic distributions, relationships, and document clusters

Topic Modeling | DGX Station at build.nvidia.com

Topic 1		Topic 2	
border	.4	space	.6
visa	.3	planet	.4
alien	.2	stars	.4
immigrant	.2	alien	.2

Document1: Topic 1: 1, Topic 2: 0

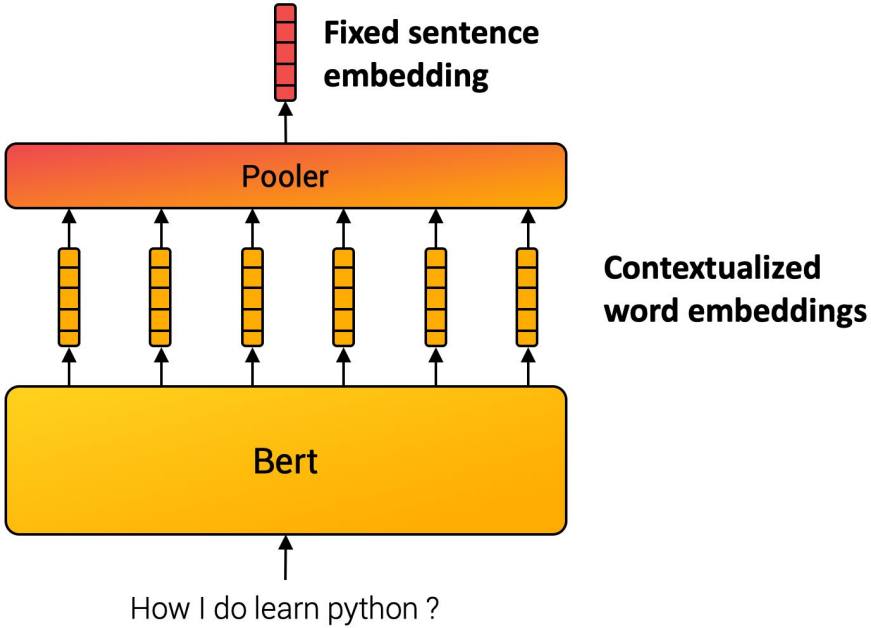
Document2: Topic 1: .1, Topic 2: .9

Document3: Topic 1: .85, Topic 2: .15

Document4: Topic 1: 0, Topic 2: 1

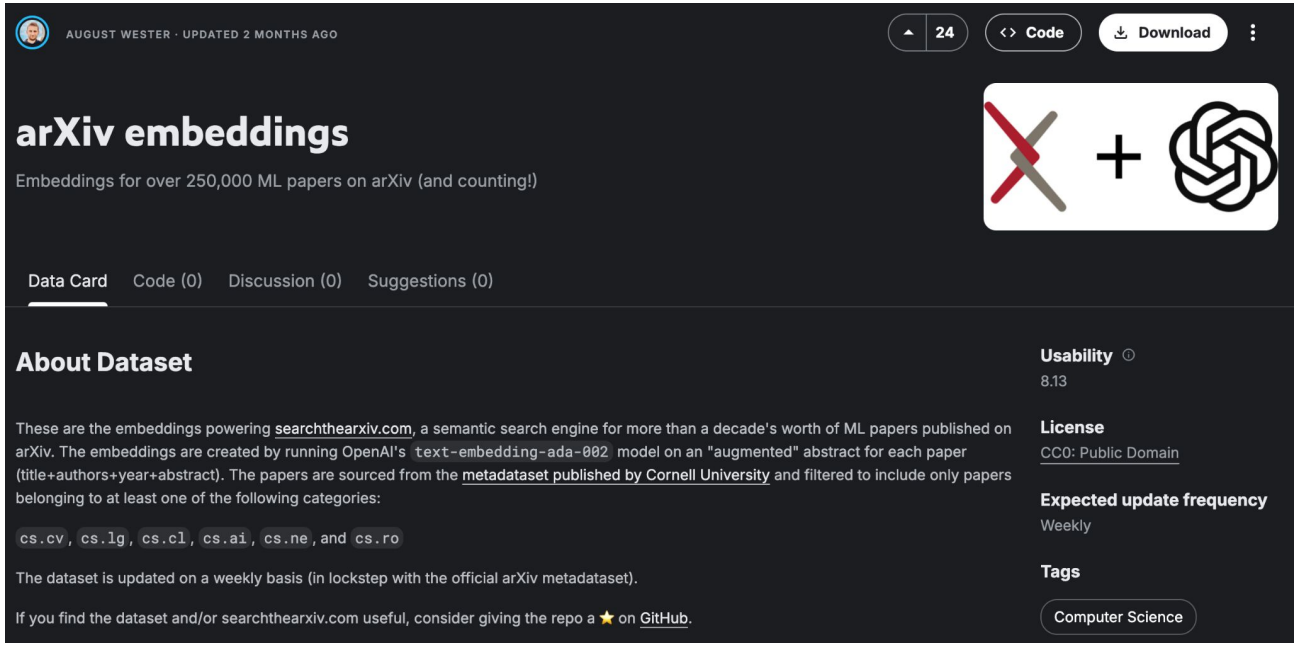
What is topic modeling?

Sentence Embeddings from BERT



Embedding Atlas

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding



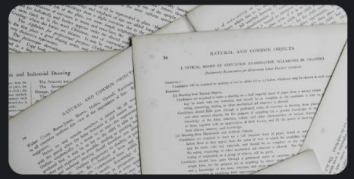
[ML Arxiv Embeddings]

SUMIT MISHRA · UPDATED 7 DAYS AGO

60 | Code | Download

arXiv Scientific Research Papers Dataset

A Curated Dataset of arXiv Papers with Titles, Summaries, Categories & Metadata.



Data Card | Code (21) | Discussion (0) | Suggestions (0)

About Dataset

Dataset Description: ArXiv Scientific Research Papers Dataset

This dataset is a **curated collection of research papers from arXiv**, covering various scientific fields such as **Artificial Intelligence, Machine Learning, computer science, mathematics and more**. It includes **titles, abstracts, categories, authors, publications and Updated dates** making it useful for various **machine learning and NLP tasks**. It can be used for several use cases given below

1. **Research Paper Classification** – Train a model to predict a paper's category based on its title or abstract.
2. **Recommendation Systems** – Suggest similar papers based on textual content or categories.
3. **Trend Analysis** – Analyze how research topics evolve.
4. **Text Summarization** – Develop models to generate concise summaries of scientific papers.
5. **Topic Modeling** – Identify hidden themes and emerging research areas.
6. **Author Impact Analysis** – Track contributions and collaborations across different domains.

Upvote if you like!! 🍺

Usability
10.00

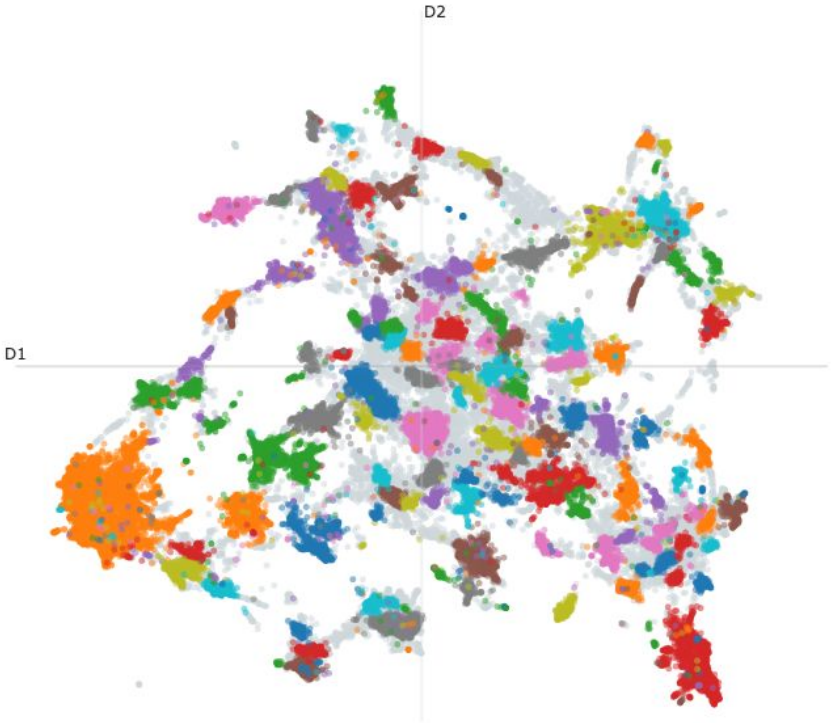
License
Apache 2.0

Expected update frequency
Quarterly

Tags

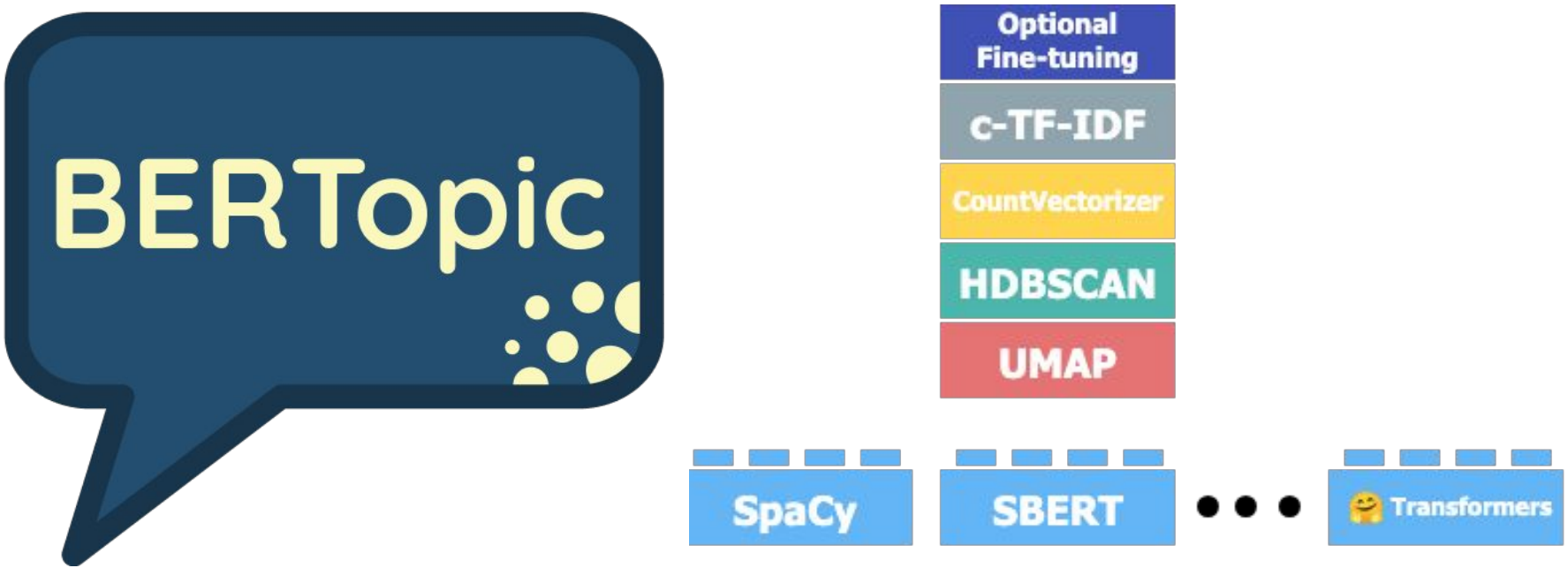
Education | Artificial Intelligence | Research | Text Mining | Transformers

Documents and Topics

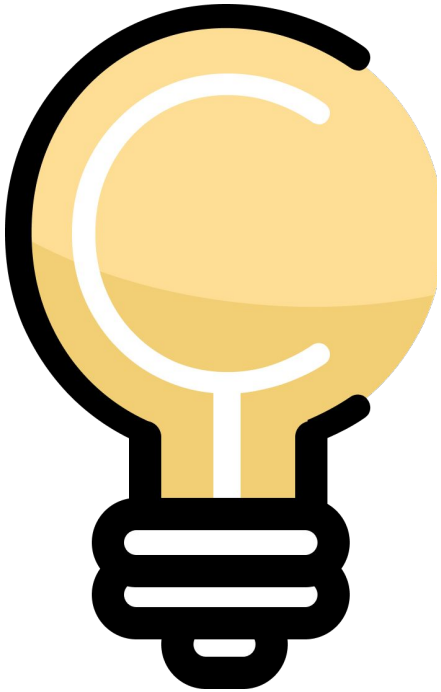


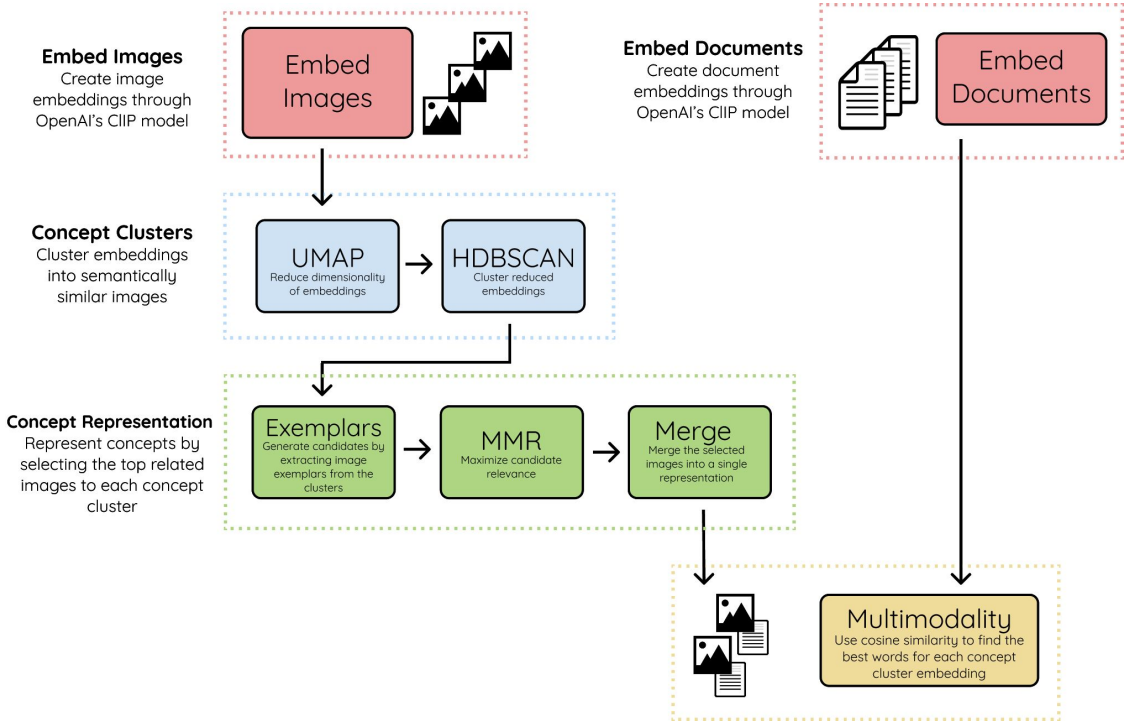
- Deep Reinforcement Learning Challenges
- Privacy-preserving federated learning
- Audio-Visual Speech Separation and Recognition
- Solving Partial Differential Equations using Neural Networks
- Adversarial Robustness in Deep Learning
- Deep Learning Theory
- Graph Embedding and Representation Learning
- Clinical Data Analysis and Missing Information Imputation
- Medical Image Segmentation for Cancer Diagnosis
- Optimization techniques for stochastic non-convex problems
- Wireless Communication Networks and Channel Estimation
- Multi-Armed Bandit Problem
- 3D Object Detection and Generation
- Generative Adversarial Networks (GANs) Training Challenges
- Recommender Systems and User Interactions
- Smart Grid Energy Management
- Causal Inference and Discovery
- Autonomous Vehicle Safety and Trajectory Planning
- Explainable Artificial Intelligence (XAI)
- Malware Detection using Machine Learning
- Fairness and Bias in Machine Learning
- Quantum Machine Learning
- Clustering Algorithms
- Time Series Forecasting
- Remote Sensing Image Classification
- Multi-Label Learning with Noisy Labels
- Distributed Machine Learning Optimization
- Few-shot Learning and Meta-Learning
- Traffic Prediction and Mobility in Urban Areas
- Anomaly Detection in Time Series using Deep Learning

[Topic Modeling with LLama 2]



https://maartengr.github.io/BERTopic/index.html



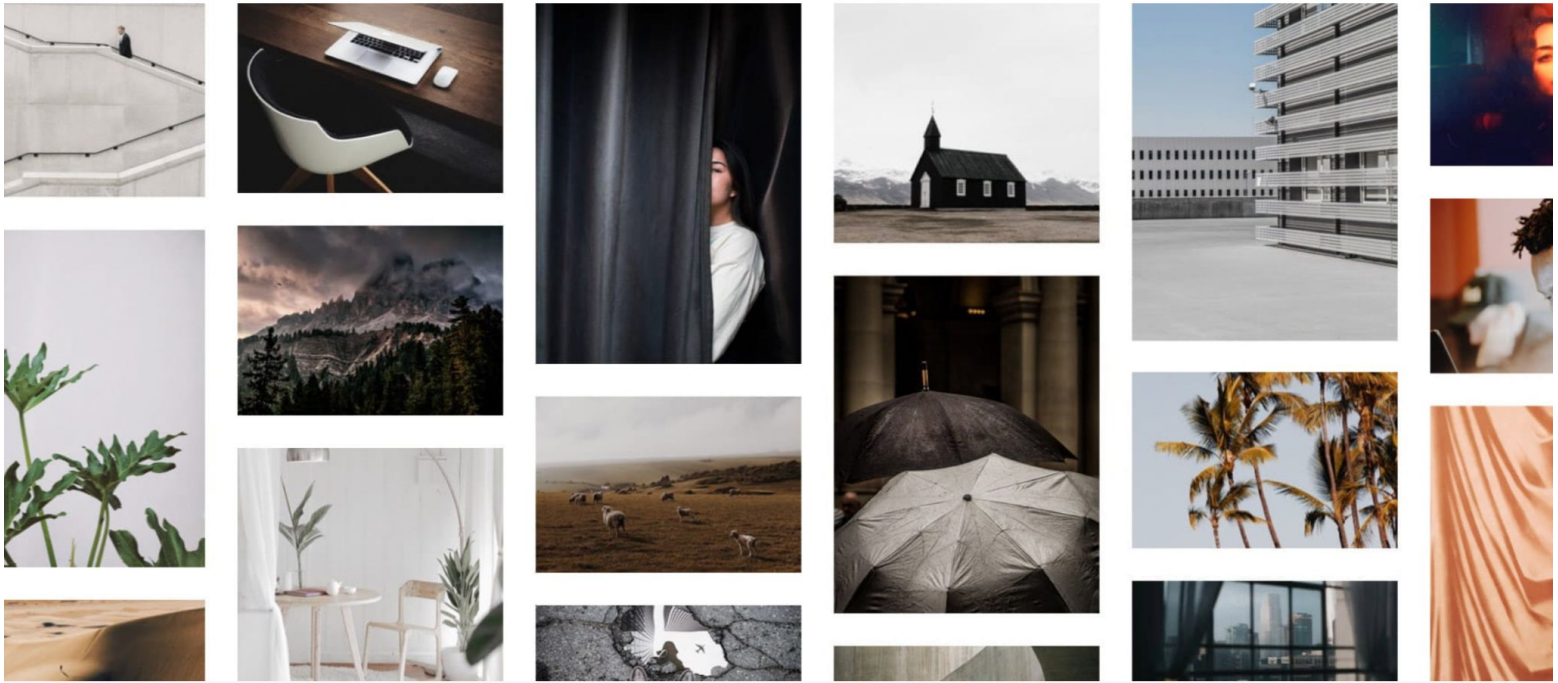


https://github.com/MaartenGr/Concept

Unsplash Developers

8.5M
photos

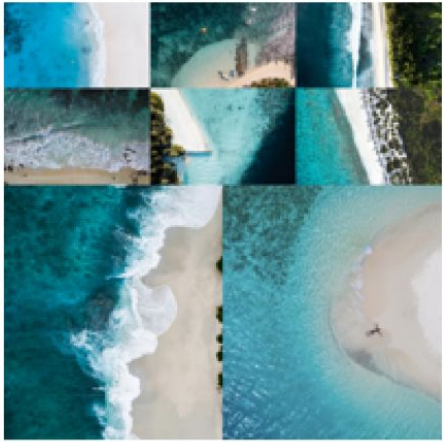
426k
photographers



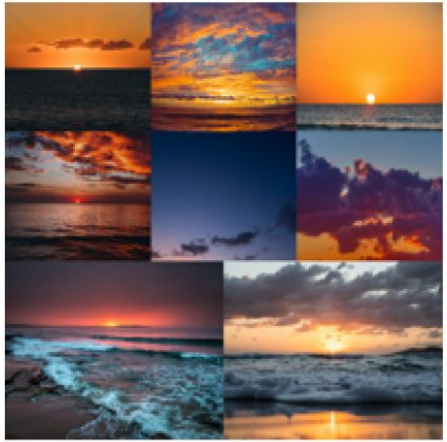
Unsplash's image collection

Concept 46:
hangfire, firewood, bonfire, fireside, fires

Concept 85:
ocean, tonga, island, islands, philippines

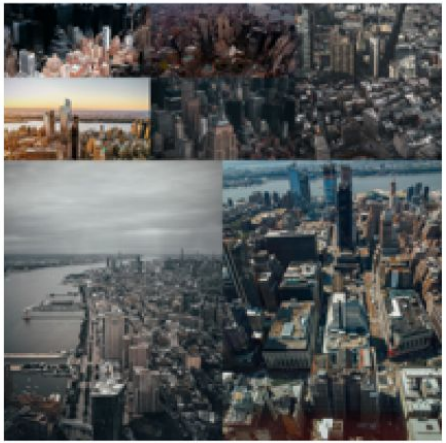


Concept 94:
sea, dawn, sunrise, sunset, subset



Concept 64:
wall, leaves, seasons, fall, garrison

Concept 74:
skyline, city, cities, nylanders, ny



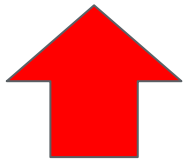
Concept 9:
insect, formicinae, nectar, insects, bee

BERTopic: Neural topic modeling with a class-based TF-IDF procedure

M Grootendorst - arXiv preprint arXiv:2203.05794, 2022 - arxiv.org

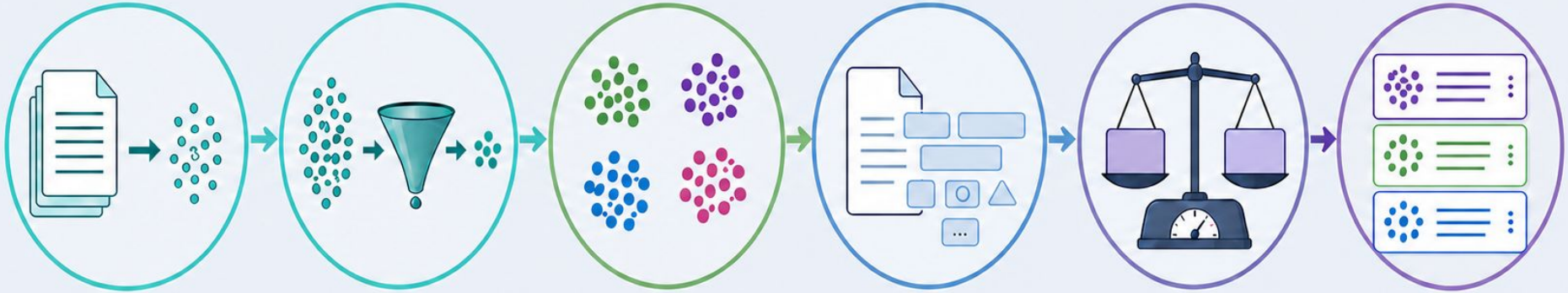
… We present **BERTopic**, a topic model that extends this process … More specifically, **BERTopic** generates document embedding … **BERTopic** generates coherent topics and remains compet…

☆ Save Cite Cited by 5482 Related articles All 5 versions View as HTML



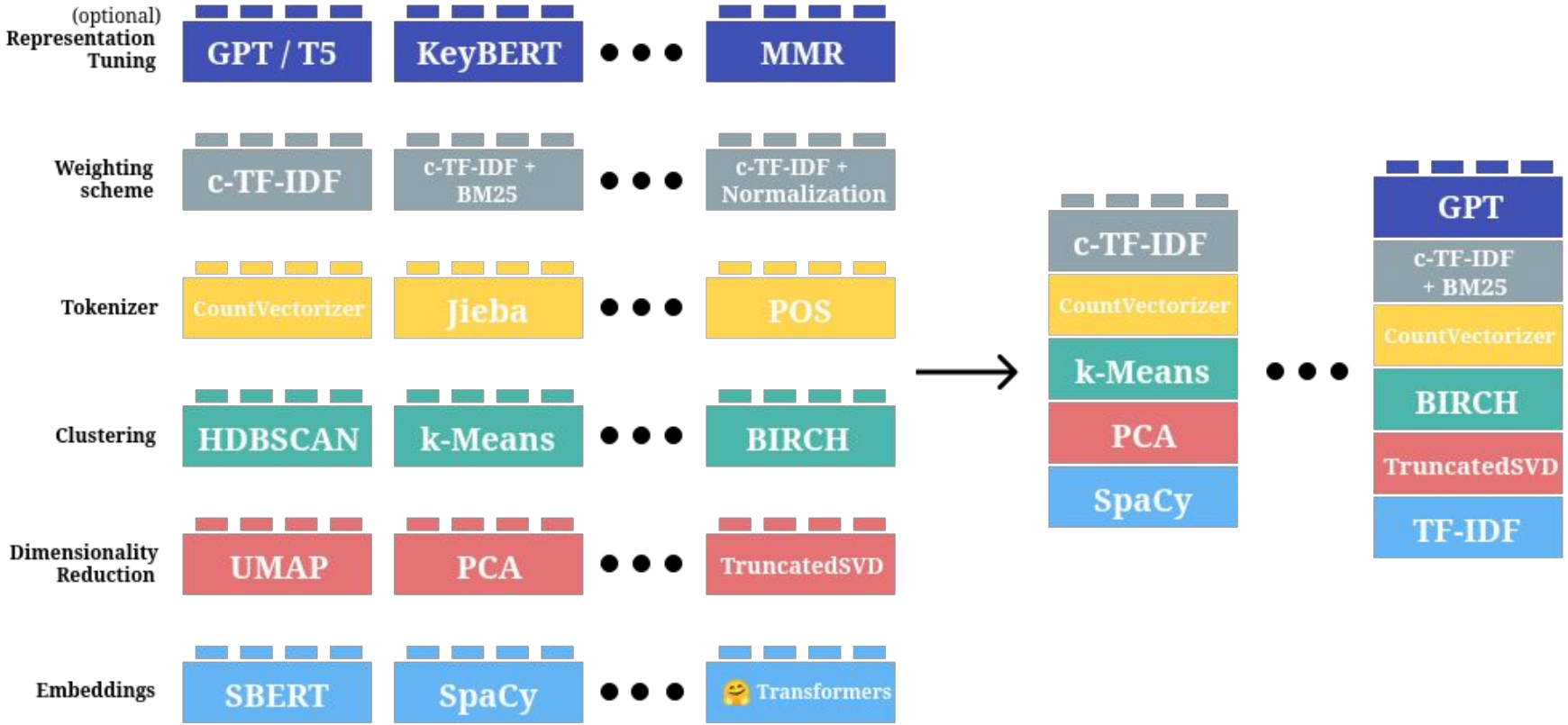
ArXiv Link

Steps for Bertopic



1. Embed documents
2. Reduce embedding dimensionality
3. Cluster reduced embeddings
4. Tokenization
5. Weigh tokens
6. Describe topics

Exploratory Swapping of Components



c-TF-IDF

For a term **x** within class **c**:

$$W_{x,c} = \| tf_{x,c} \| \times \log\left(1 + \frac{A}{f_x}\right)$$

$tf_{x,c}$ = frequency of word **x** in class **c**

f_x = frequency of word **x** across all classes

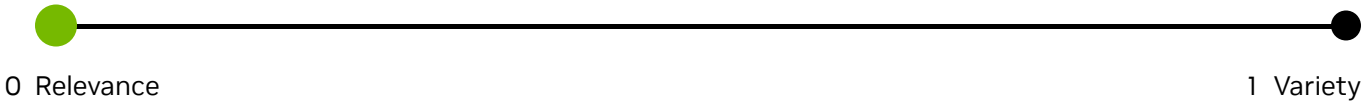
A = average number of words per class

Keyword extraction in BERTopic

MMR: Choose what fits, skip what repeats

Close to target + **Different from picks**

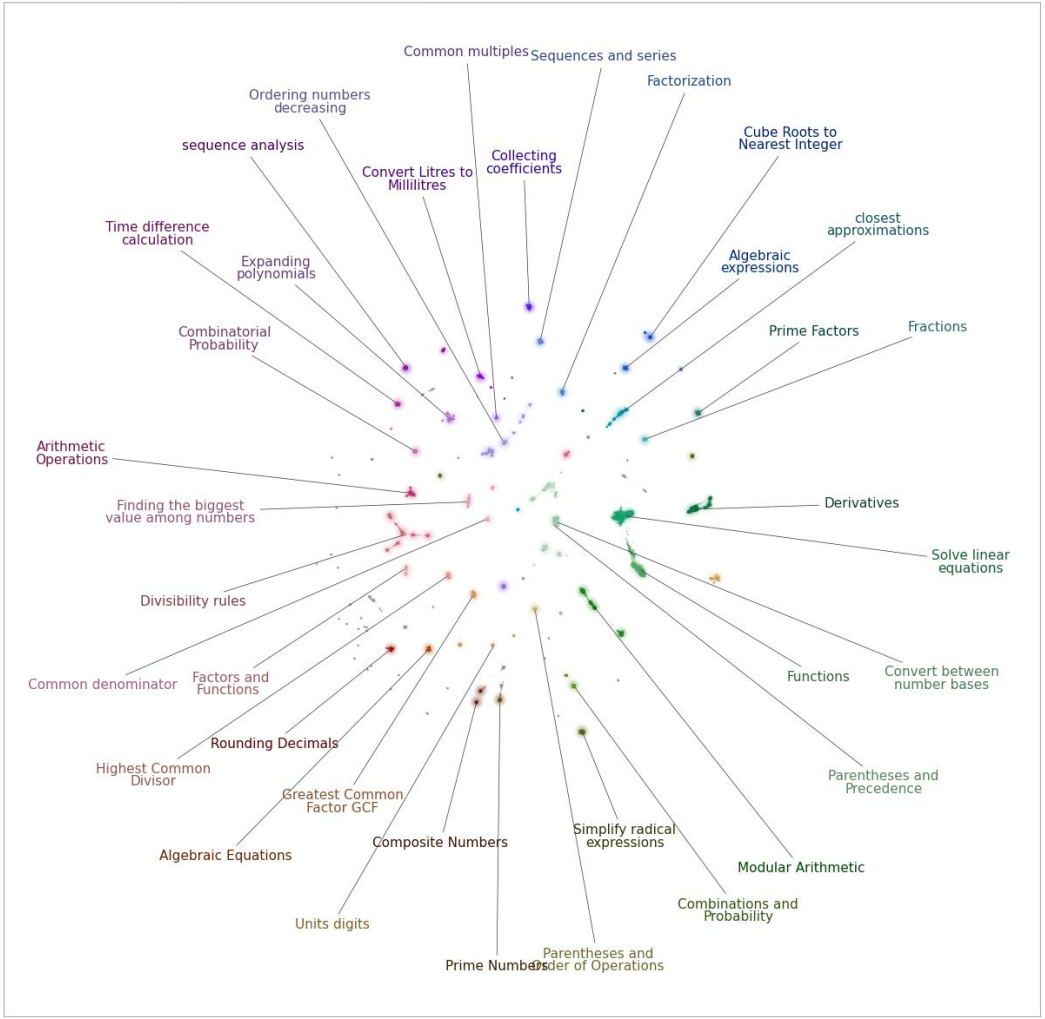
score = relevance − redundancy



Source: BERTopic MMR documentation

The Three Pillars of BERTopic

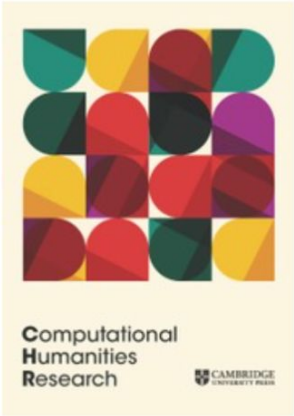
Topic Modeling + Exploration + Visualization



Dataset curation for retrieval-augmented generation

A curated dataset of high-school mathematics questions

Google DeepMind's Mathematics Dataset



Computational Humanities Research

Topic-RAG for historical newspapers: Enhancing information retrieval in humanities research through topic-based retrieval-augmented generation

Part of: CHR Expanding the Toolkit: Large Language Models in Humanities Research

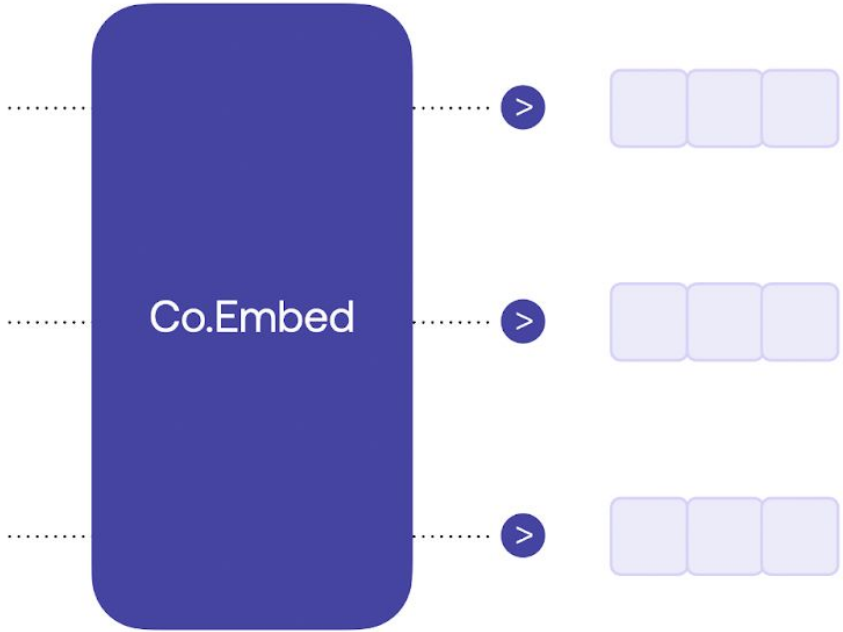
Published online by Cambridge University Press: **18 November 2025**

Keerthana Murugaraj, Salima Lamsiyah, Marten During and Martin Theobald

Show author details

Article | Figures | Rapid Responses | Metrics

Topic-RAG for historical newspapers: Enhancing information retrieval in humanities research through topic-based retrieval-augmented generation | Computational Humanities Research | Cambridge Core



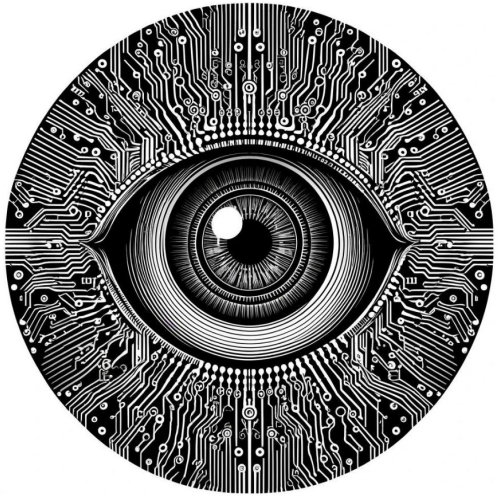
https://cohere.com/blog/embedding-archives-wikipedia

```
from datasets import load_dataset
docs = load_dataset(f"Cohere/wikipedia-22-12-simple-embeddings", split="train")
```

Review Questions

- How do we create text or image embeddings?
- How do we reduce the dimensionality of embeddings?
- What is the purpose of clustering the embeddings?
- What do we tokenize after we have clustered documents?
- Why do we use c-TF-IDF?

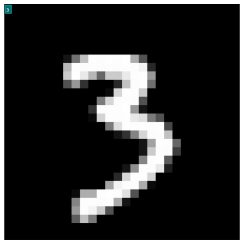
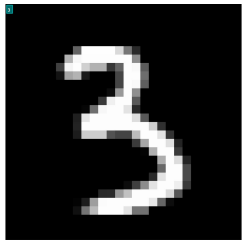
Image Embeddings



Learning goals

- Understand dense vector embeddings as learned representations
- Describe how image embeddings are produced in neural networks
- Extract image embeddings out of a pretrained Resnet50
- Inspect image neighborhoods using the cosine similarity metric

Sparse vs dense vector representations

$\mathbf{x}_1 =$  $\mathbf{x}_2 =$ 

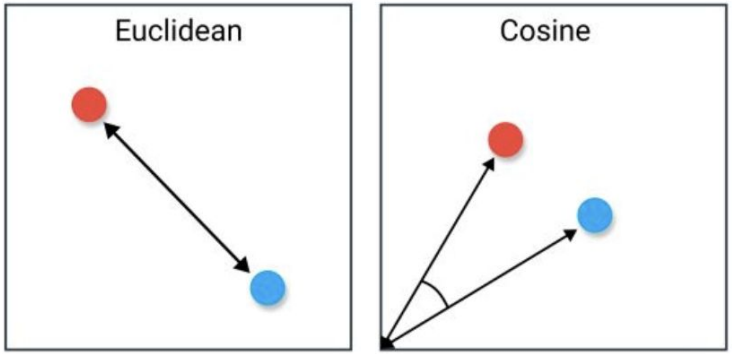
28 x 28 pixels = 784 values

$$\mathbf{x}_1\text{sparse} = \mathbf{x}_2\text{sparse} = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}^\top$$

$$\mathbf{x}_2\text{dense} = \begin{bmatrix} 0.10 & -0.75 & 0.35 & 0.41 & -0.20 & 0.89 & -0.60 & 0.03 & 0.59 & -0.50 \end{bmatrix}^\top$$

$$\mathbf{x}_1\text{dense} = \begin{bmatrix} 0.12 & -0.85 & 0.37 & 0.44 & -0.22 & 0.90 & -0.63 & 0.01 & 0.58 & -0.49 \end{bmatrix}^\top$$

Common metrics to compare embeddings

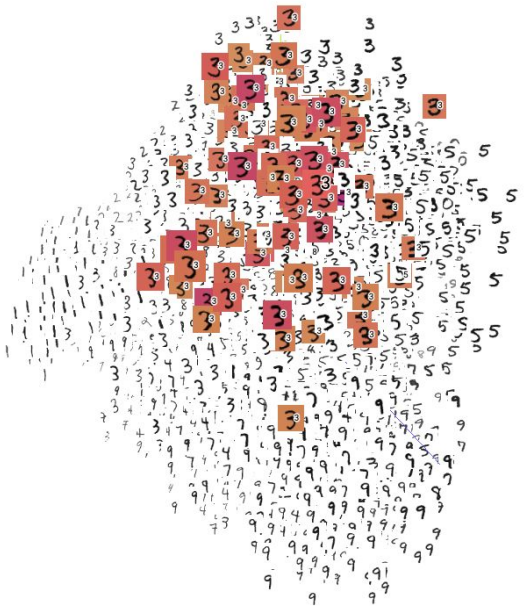


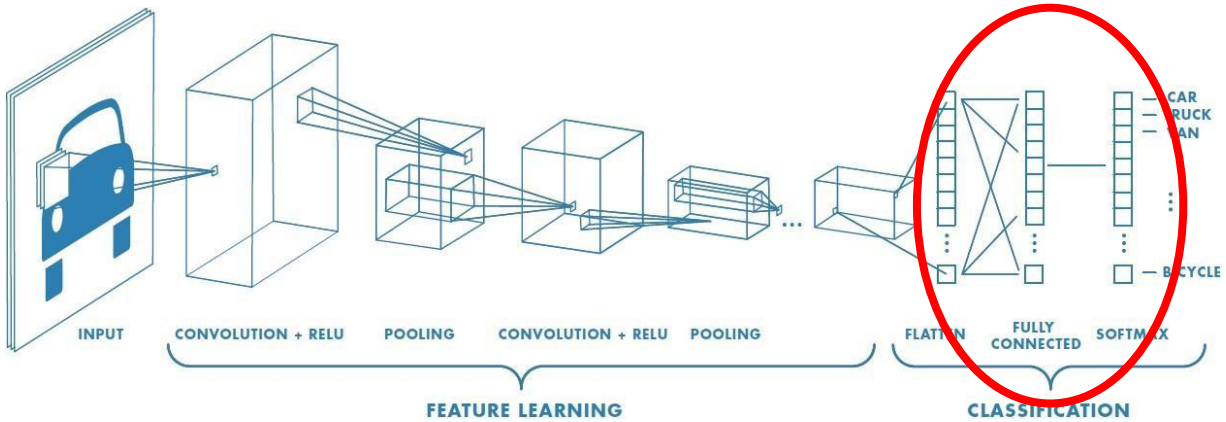
$$d_{\text{euclid}}(x_1, x_2) = \sqrt{\sum_{i=1}^{n}(x_{1i} - x_{2i})^2}$$

$$\text{cosine similarity}(x_1, x_2) = \frac{x_1 \cdot x_2}{\|x_1\| \, \|x_2\|}$$

Embeddings as a byproduct of training

When we train a neural network, we make it **embed** inputs that are semantically similar, close to each other





Dense vector embeddings can be produced by flattening, pooling, or feeding (learned) features to linear layers

Source of image (try exploring interactively)

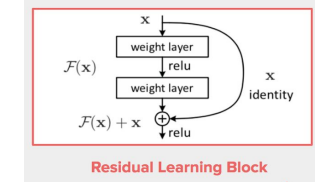
Extract embeddings from a pretrained Resnet50

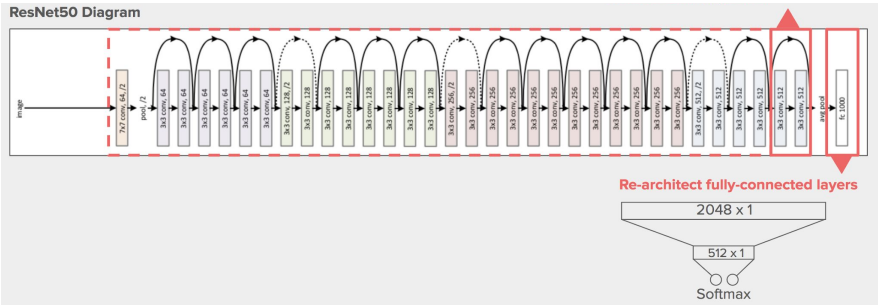
```python
import torch
from torchvision import models

# Load pre-trained ResNet50 weights and transforms
weights = models.ResNet50_Weights.IMAGENET1K_V2
model = models.resnet50(weights=weights)
# Preprocess the image using the transformations from the imagenet dataset
preprocess = weights.transforms()

# Remove final Imagenet classification layer, keep 2048 dense vector
# Take time to unpack this syntax in the practical notebook
embedder = torch.nn.Sequential(*list(model.children())[:-1])

# Disable BatchNormalization
embedder.eval()
# Disable gradient computation
with torch.inference_mode():
    # unsqueeze(0) adds the batch size for inference
    # squeeze() removes the singleton dimensions from the output vector
    embedding = embedder(preprocess(image).unsqueeze(0)).squeeze()
```





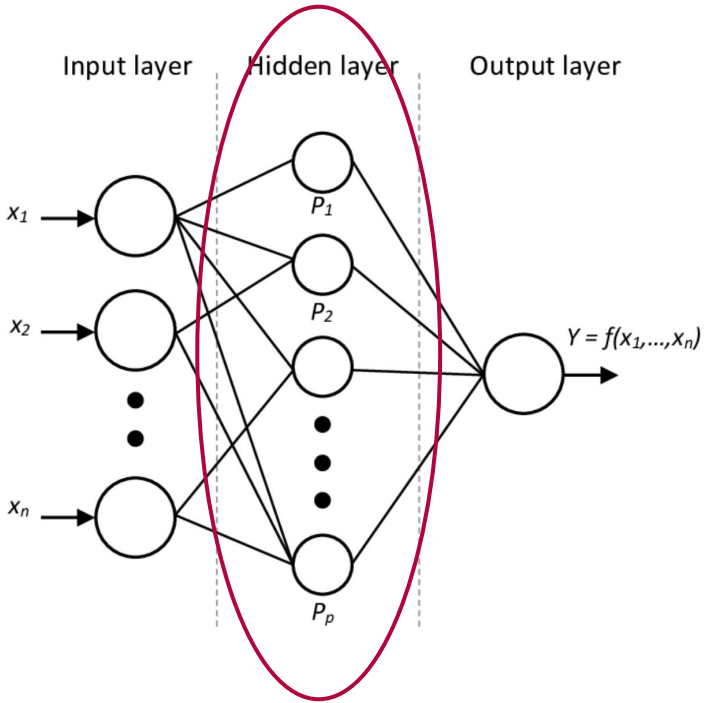
Approaches to produce embeddings with neural networks

```
import torch.nn as nn

# Approach 1: Direct embedding
linear_embedder = nn.Sequential(
    nn.Flatten(),  # 784
    nn.Linear(784, 10)
)

# Approach 2: Convolution and then fully connected layer
conv_linear_embedder = nn.Sequential(
    nn.Conv2d(1, 32, kernel_size=3),  # 26x26x32
    nn.ReLU(),
    nn.Conv2d(32, 64, kernel_size=3),  # 24x24x64
    nn.ReLU(),
    nn.Flatten(),  # 24*24*64
    nn.Linear(24*24*64, 10)
)

# Approach 3: Global pooling after convolutions
conv_pool_embedder = nn.Sequential( # MNIST input is size 28x28
    nn.Conv2d(1, 32, kernel_size=3),  # 26x26x32
    nn.ReLU(),
    nn.Conv2d(32, 10, kernel_size=3),  # 24x24x10
    nn.ReLU(),
    nn.AdaptiveAvgPool2d((1, 1)),  # 1x1x10
    nn.Flatten()  # 10
)
```



$$L_{L1} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

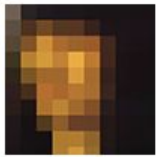
README.md

Art Recommendation system

This is the repository of a portfolio project at DSR. This project aims to identify similar images using pre-trained computer vision networks. For an explanation of the technology see the technology section.

Contributors

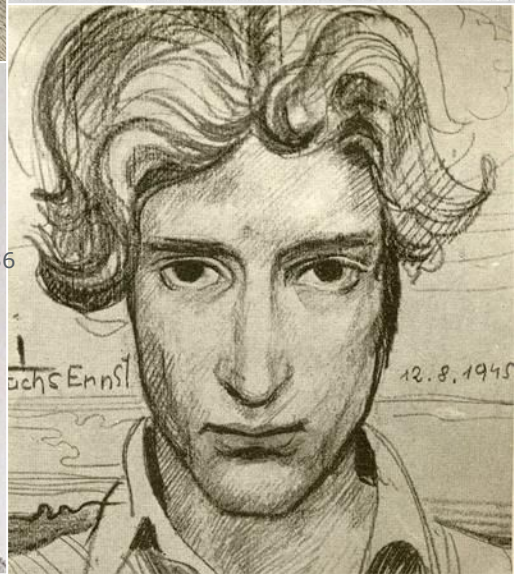
- Catarina Ferreira
- Gargi Maheshwari

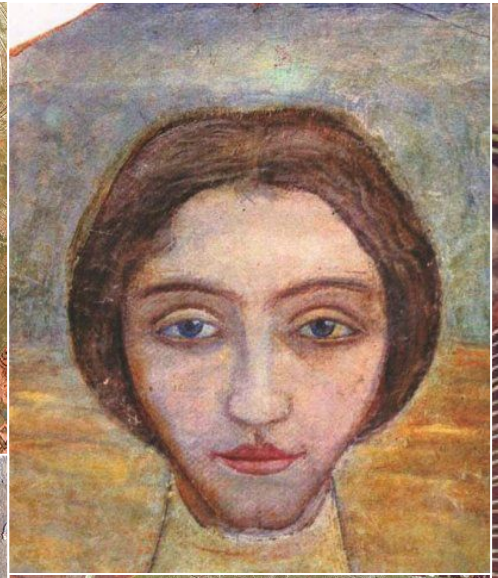


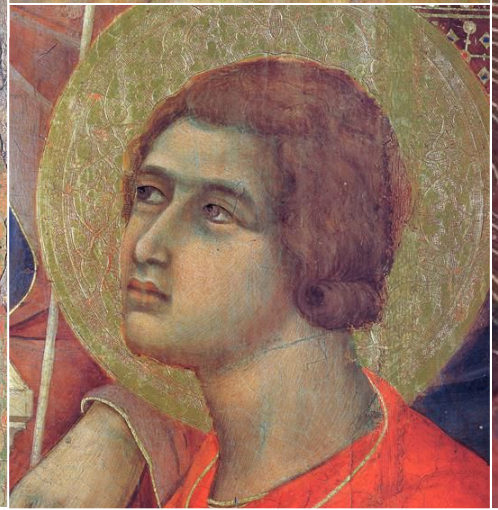
WIKIART
VISUAL ART ENCYCLOPEDIA

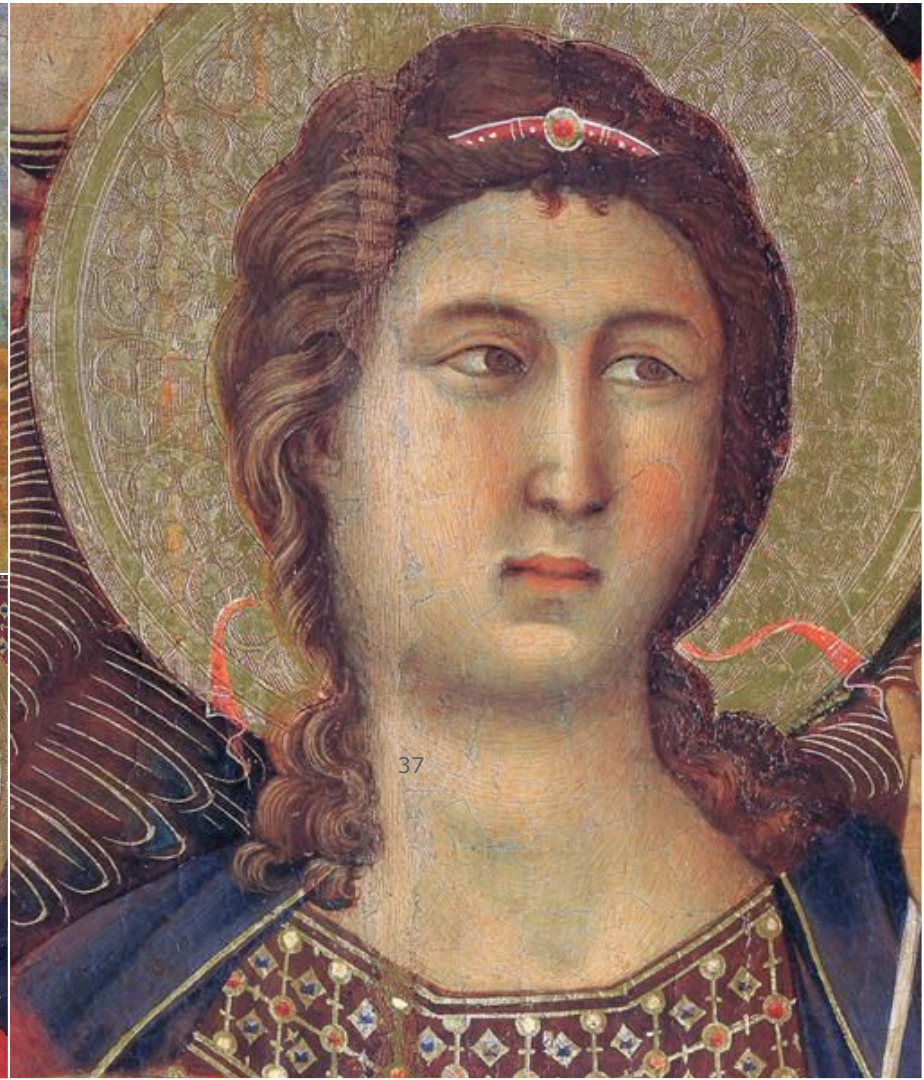
https://github.com/gargimaheshwari/Wikiart-similar-art





12.8.1945







Application: differential diagnosis

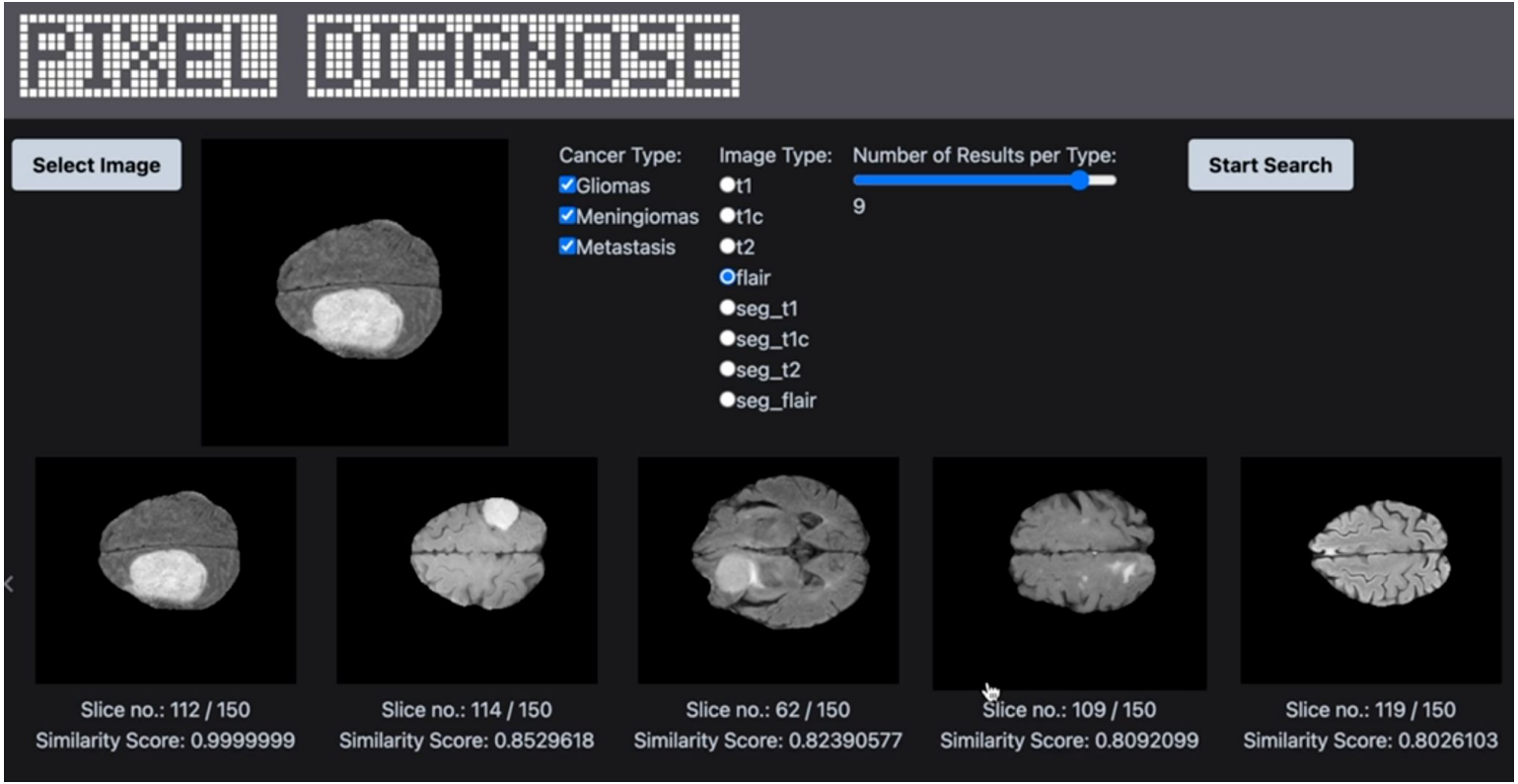


Image from https://pixel-diagnose.github.io/

Summary

Embeddings are learned vector representations

- Neural networks learn to map similar images to similar vectors through training
- We measure the similarity of embeddings using metrics like cosine similarity and Euclidean distance

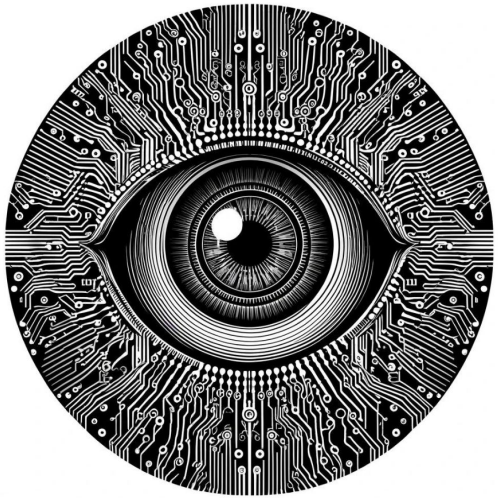
There are different architectural approaches to create embeddings

- We can use linear layers, or flattened and pooled convolutions to produce embeddings
- Imagenet-pretrained models produce rich representations in their embeddings

Embeddings have multiple applications in semantic search and dataset curation

- They can be used for reverse image search, recommendation systems, and deduplication

Contrastive Language-Image Pretraining (CLIP)



Learning goals

- Understand how CLIP models are trained
- Create image and text embeddings with a pretrained CLIP model

CLIP: ‘Contrastive Language Image Pretraining’

Learning Transferable Visual Models From Natural Language Supervision

Alec Radford [*1] **Jong Wook Kim** [*1] **Chris Hallacy** [1] **Aditya Ramesh** [1] **Gabriel Goh** [1] **Sandhini Agarwal** [1]
Girish Sastry [1] **Amanda Askell** [1] **Pamela Mishkin** [1] **Jack Clark** [1] **Gretchen Krueger** [1] **Ilya Sutskever** [1]

- Connects text and images in a shared embedding space
- Created by OpenAI in 2021
- Trained on **400M image-textual description pairs** scraped from the internet
- Predicts the most relevant text snippet given an image, enabling “zero-shot classification”

Aligning text and image embeddings

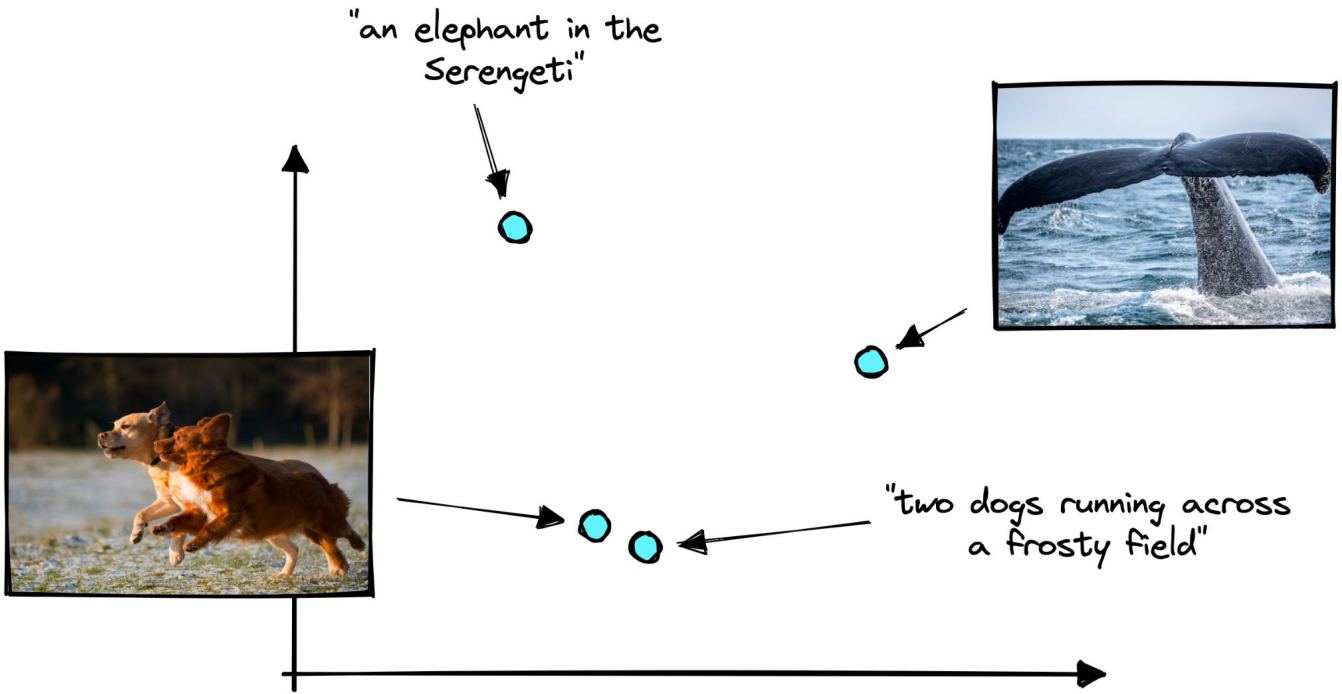
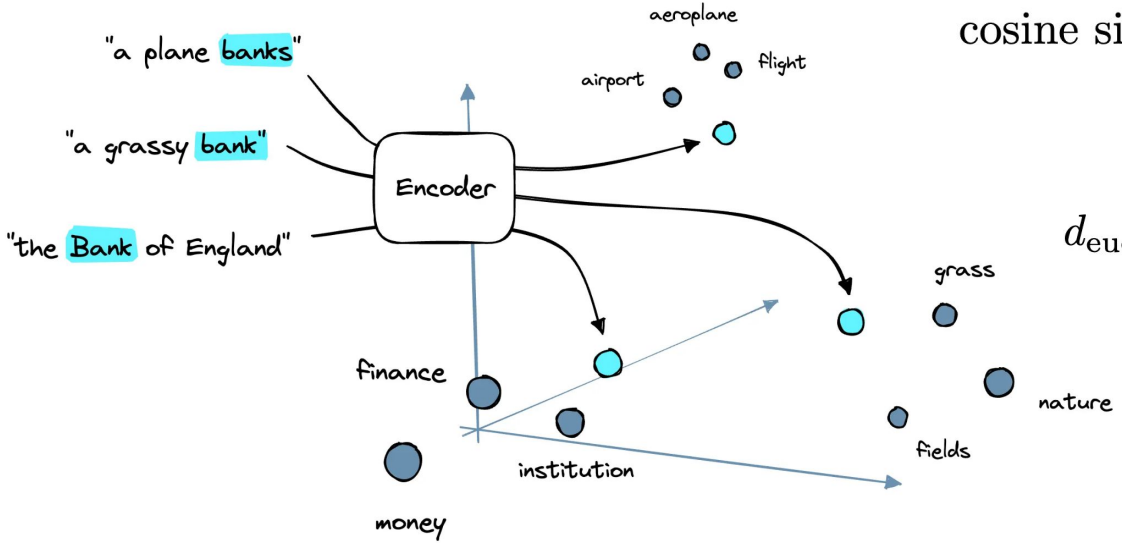


Image from https://www.pinecone.io/learn/series/image-search/clip/

Text encoders

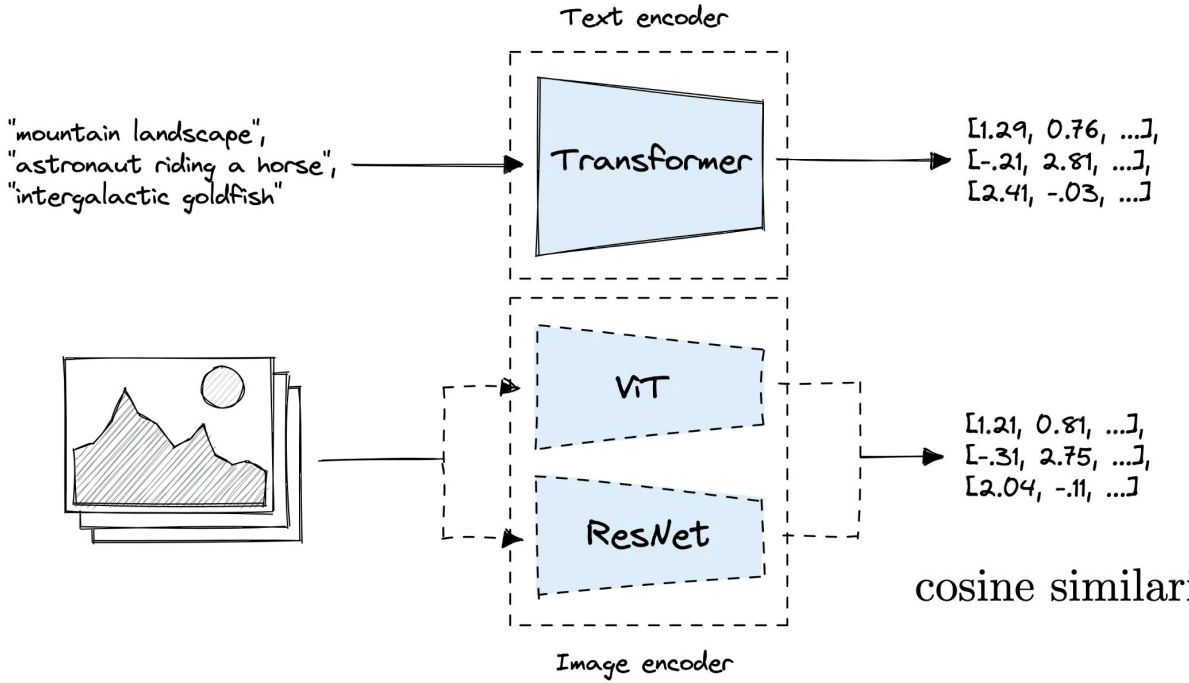


$$\text{cosine similarity}(x_1, x_2) = \frac{x_1 \cdot x_2}{\|x_1\| \, \|x_2\|}$$

$$d_{\text{euclid}}(x_1, x_2) = \sqrt{\sum_{i=1}^{n} (x_{1i} - x_{2i})^2}$$

Image from https://www.pinecone.io/learn/series/image-search/vision-transformers/

CLIP's architecture



Both encoders map inputs to 512 dimensional embeddings

$$\text{cosine similarity}(x_1, x_2) = \frac{x_1 \cdot x_2}{\|x_1\| \, \|x_2\|}$$

Image from https://www.pinecone.io/learn/series/image-search/clip/

Maximizing cosine similarity of matching text and image embeddings

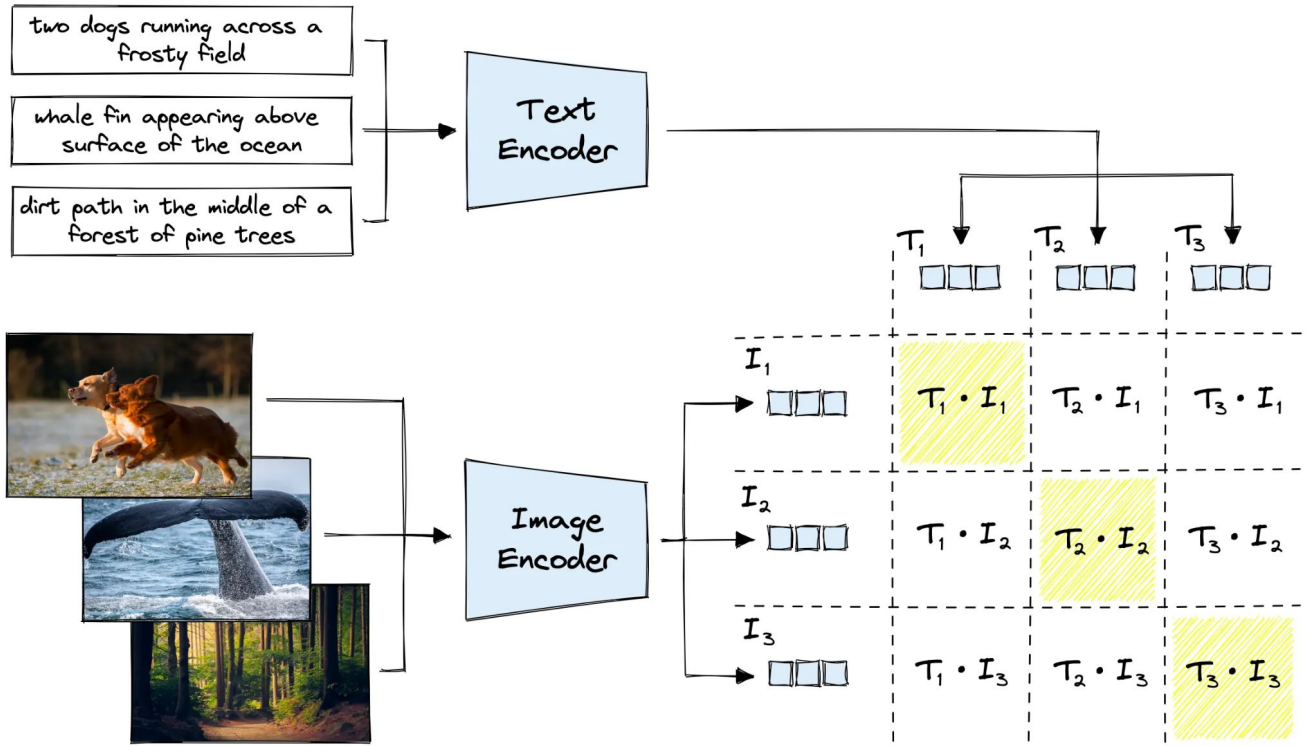


Image from https://www.pinecone.io/learn/series/image-search/clip/

Training algorithm

```
# image_encoder - ResNet or Vision Transformer
# text_encoder  - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l]       - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t             - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T)  #[n, d_t]

# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss   = (loss_i + loss_t)/2
```

Figure 3. Numpy-like pseudocode for the core of an implementation of CLIP.

	T_1	T_2	T_3	...	T_N
I_1	$I_1 \cdot T_1$	$I_1 \cdot T_2$	$I_1 \cdot T_3$	...	$I_1 \cdot T_N$
I_2	$I_2 \cdot T_1$	$I_2 \cdot T_2$	$I_2 \cdot T_3$	...	$I_2 \cdot T_N$
I_3	$I_3 \cdot T_1$	$I_3 \cdot T_2$	$I_3 \cdot T_3$	...	$I_3 \cdot T_N$
⋮	⋮	⋮	⋮	⋱	⋮
I_N	$I_N \cdot T_1$	$I_N \cdot T_2$	$I_N \cdot T_3$	...	$I_N \cdot T_N$

$$H(p,q) = -\sum_i p(i) \log q(i)$$

$$H(q,p) = -\sum_i q(i) \log p(i)$$

$$\text{symmetric CE loss} = \frac{H(p,q) + H(q,p)}{2}$$

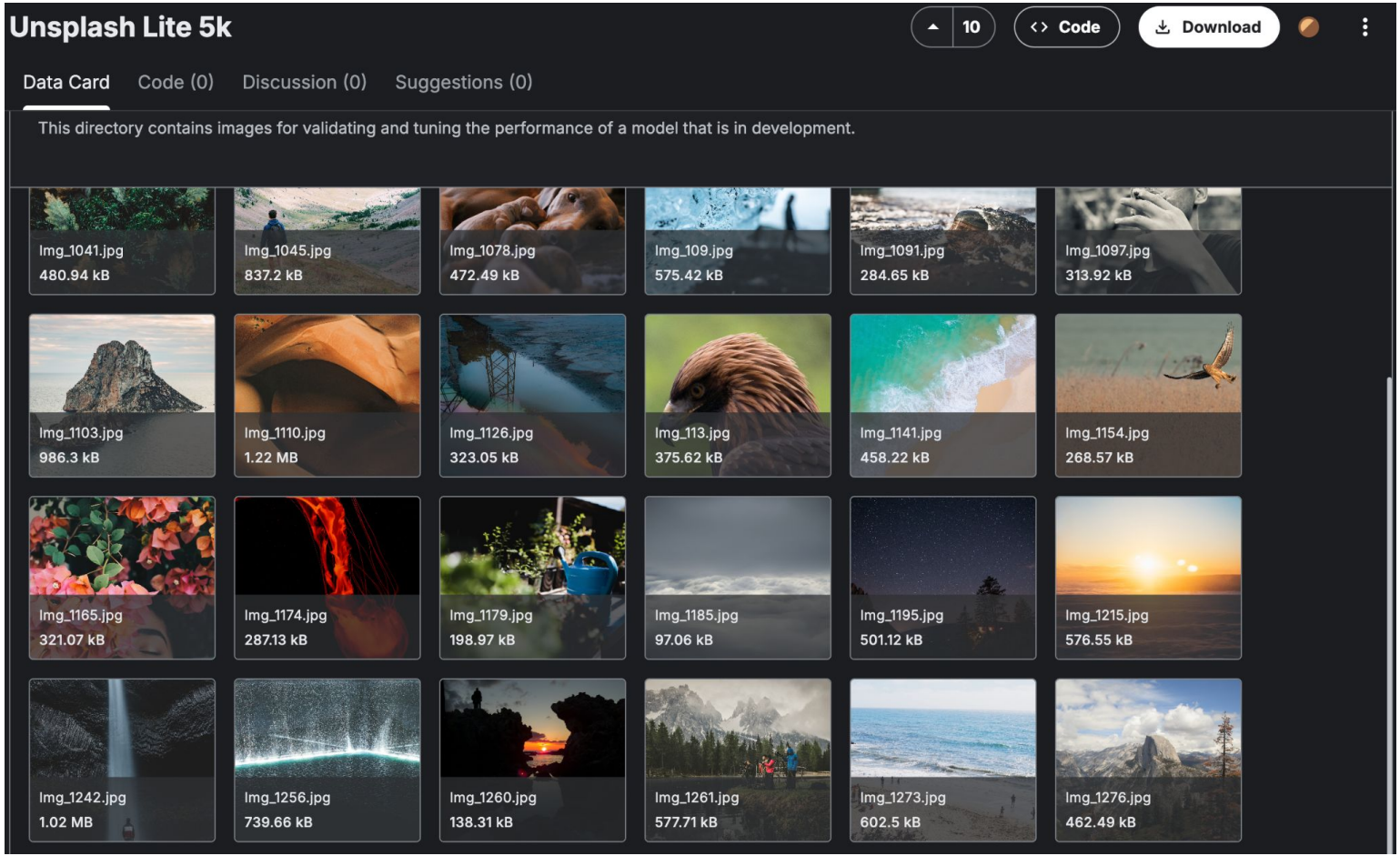
NVIDIA’s C-RADIO for Text ↔ Image

Text Prompt: "shoe"

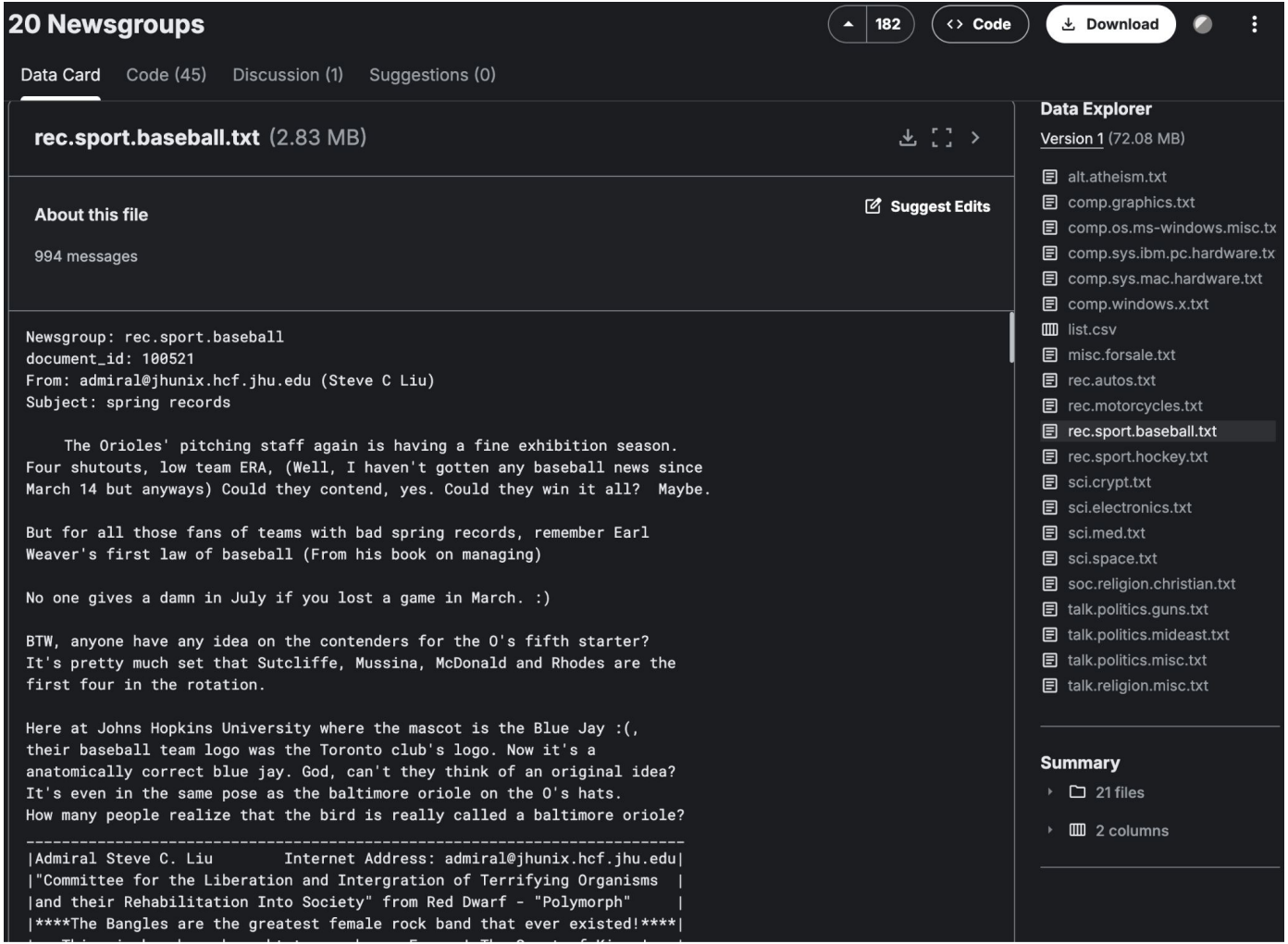
Text Prompt: "helmet"

https://huggingface.co/nvidia/C-RADIOv4-H

C-RADIOv4 (Tech Report)

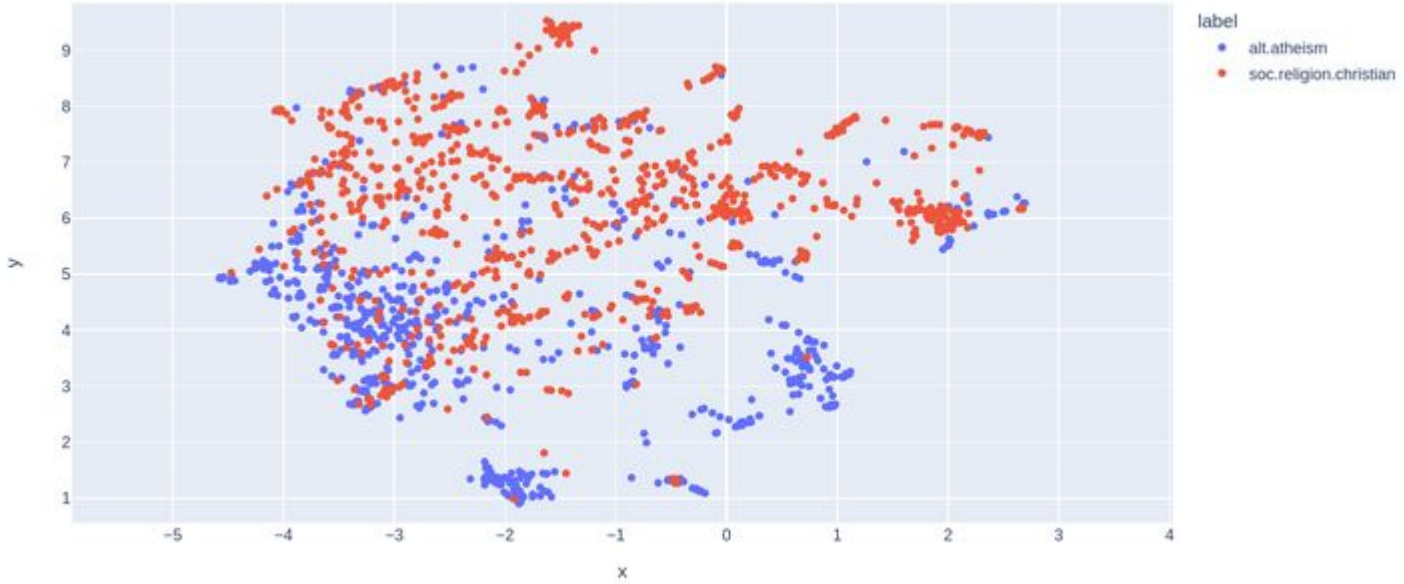


[Kaggle Unsplash Lite 5k]

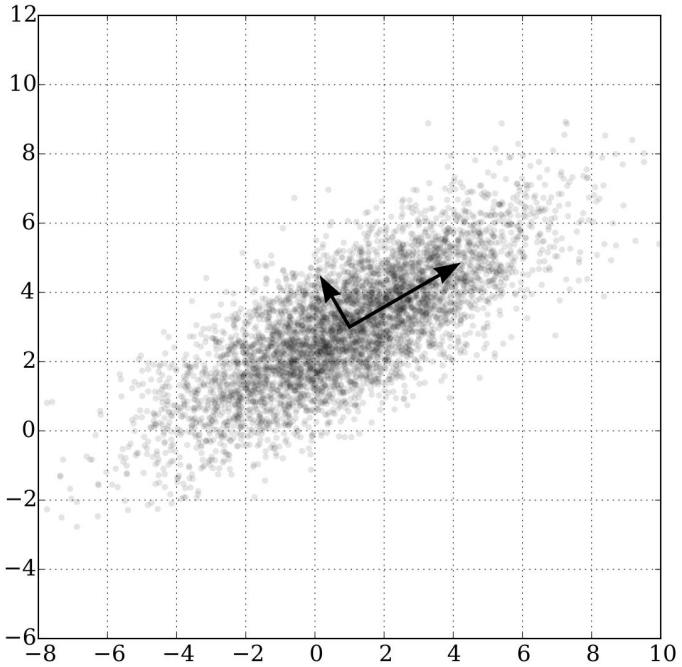


[Kaggle - 20 Newsgroups]

Dimensionality Reduction

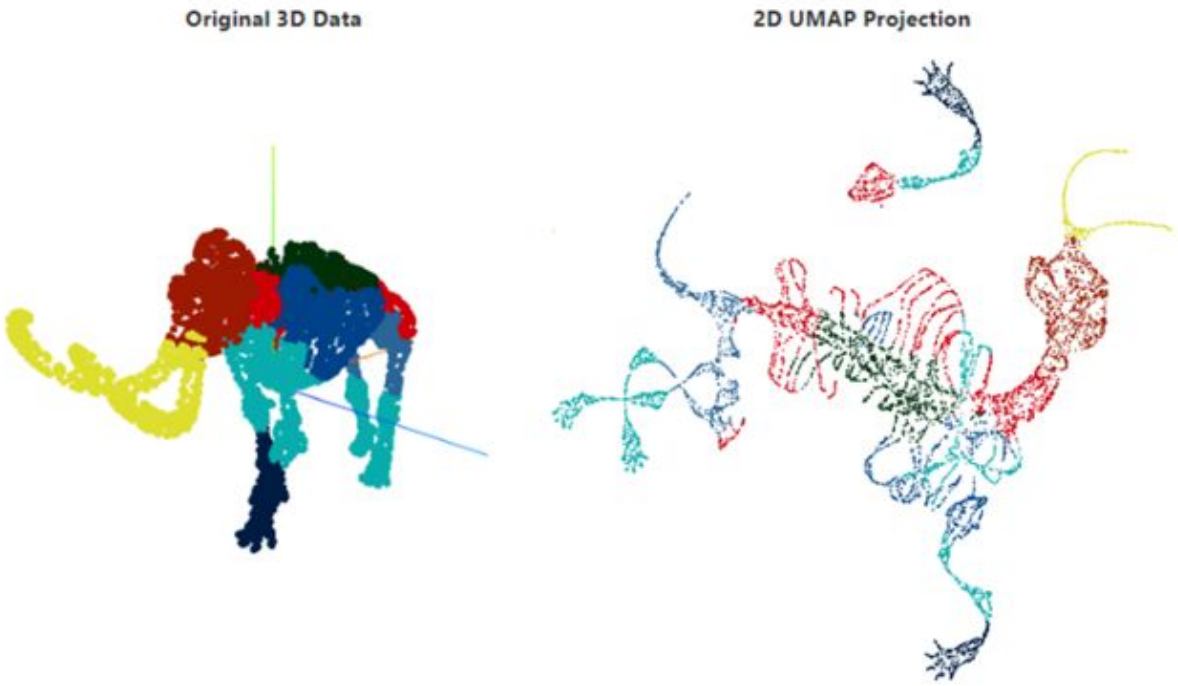


Principal Component Analysis



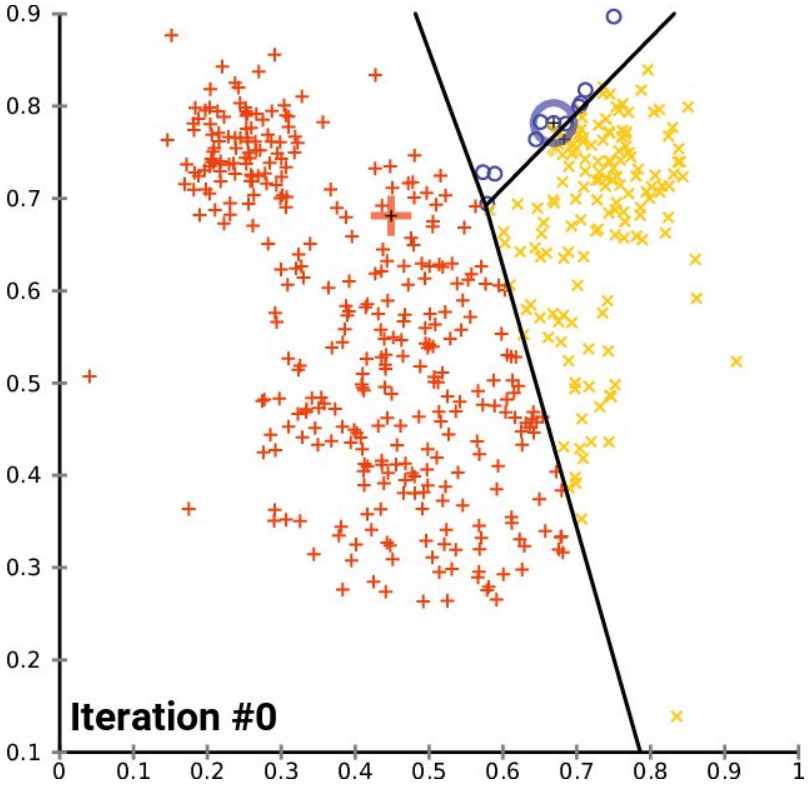
https://en.wikipedia.org/wiki/Principal_component_analysis

UMAP

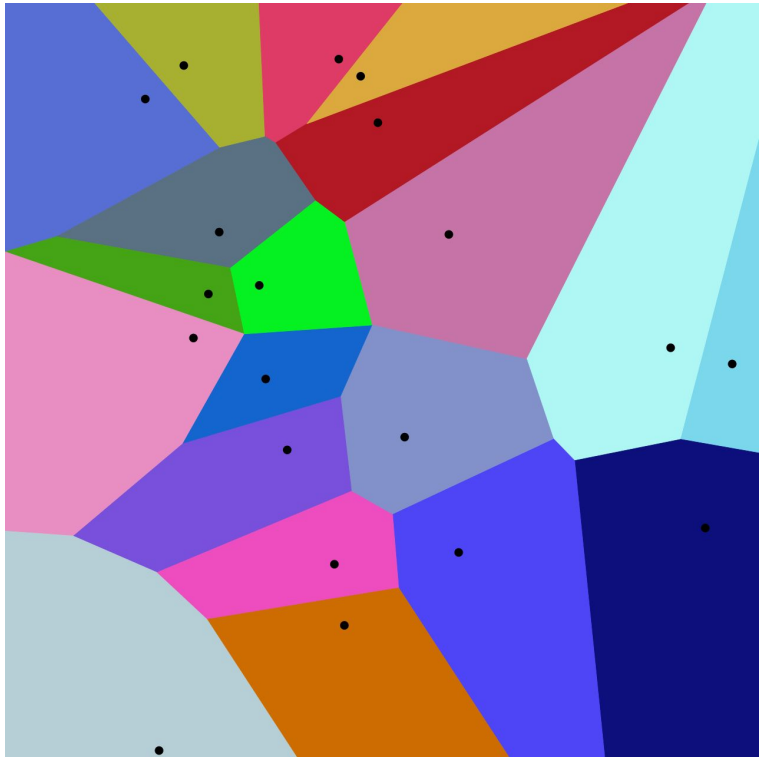


UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction (McInnes et al., 2018)

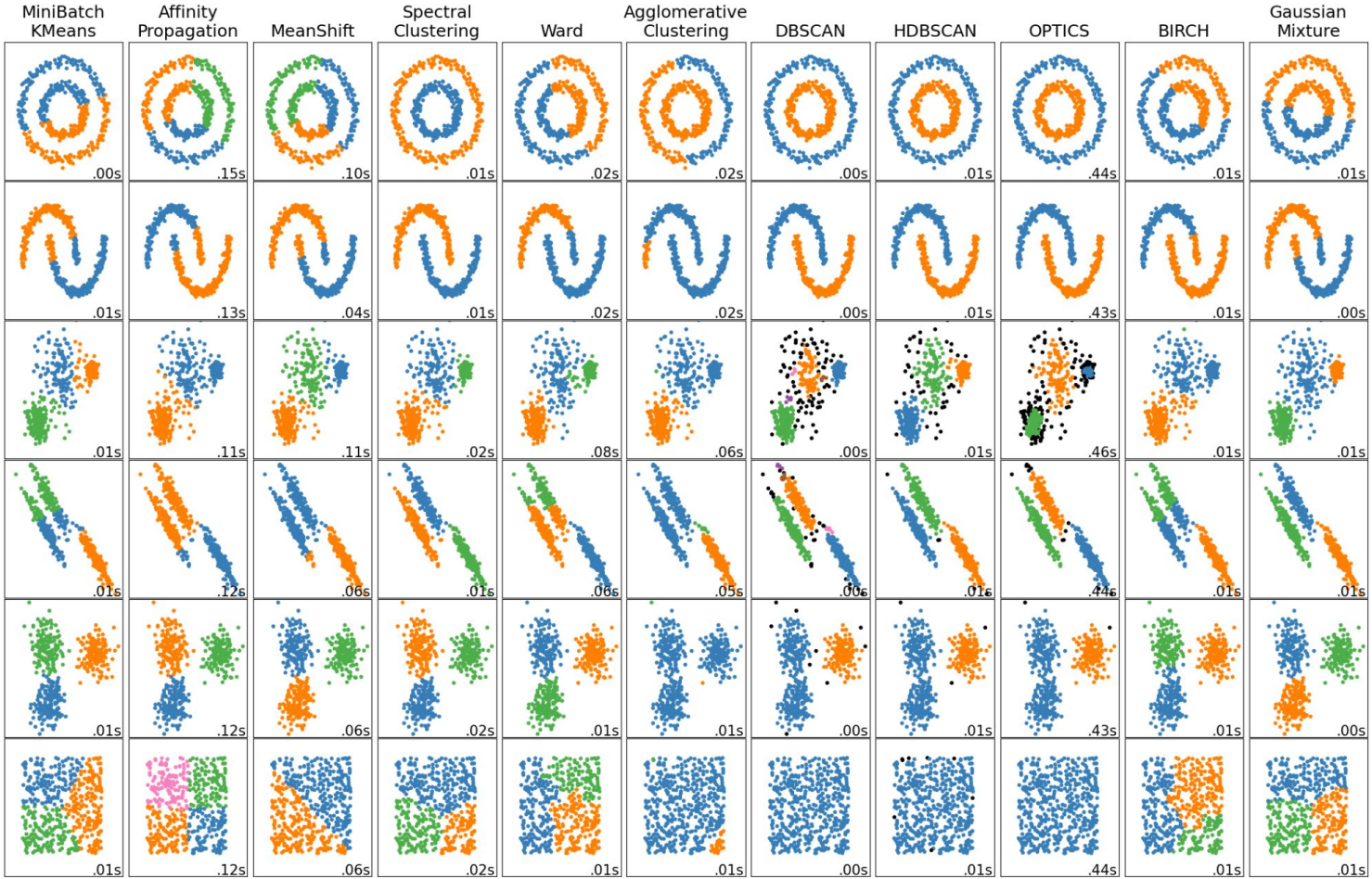
K-means clustering



k-means clustering - Wikipedia

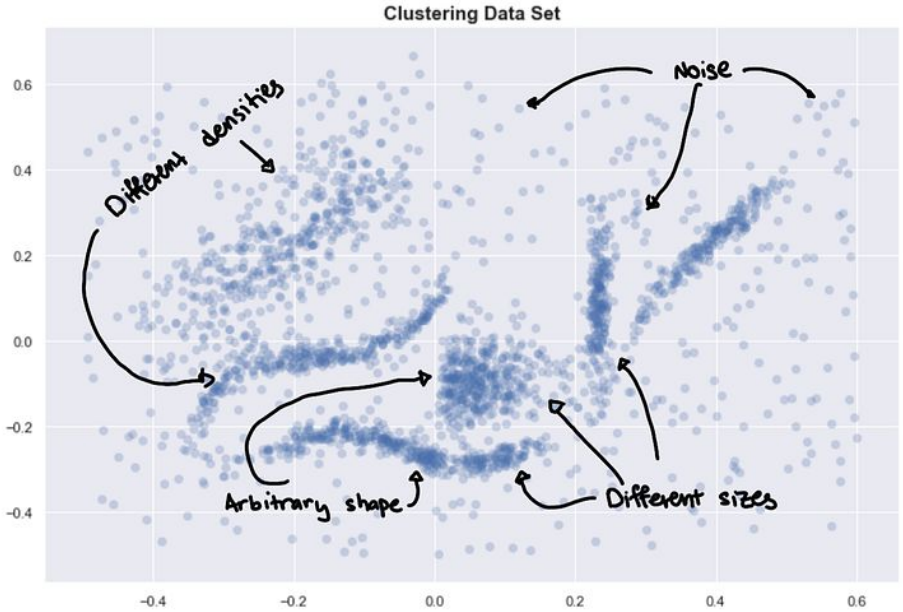


Voronoi diagram - Wikipedia



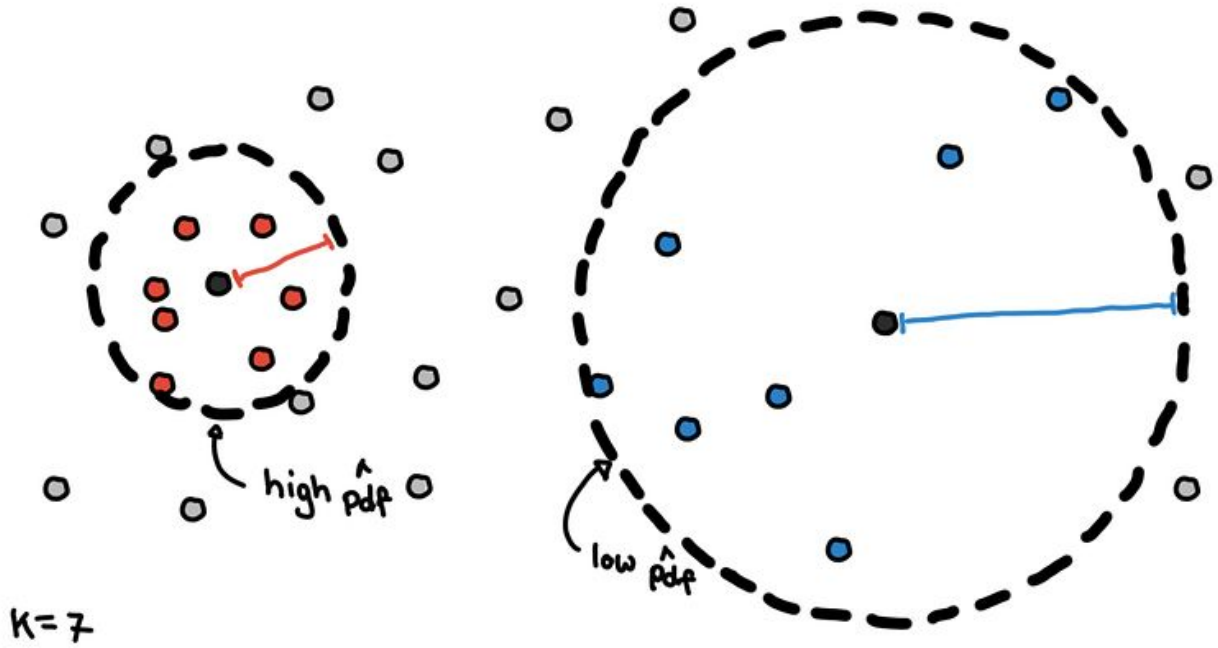
Comparing different clustering algorithms on toy datasets — scikit-learn 1.9.0 documentation

Clustering based on density



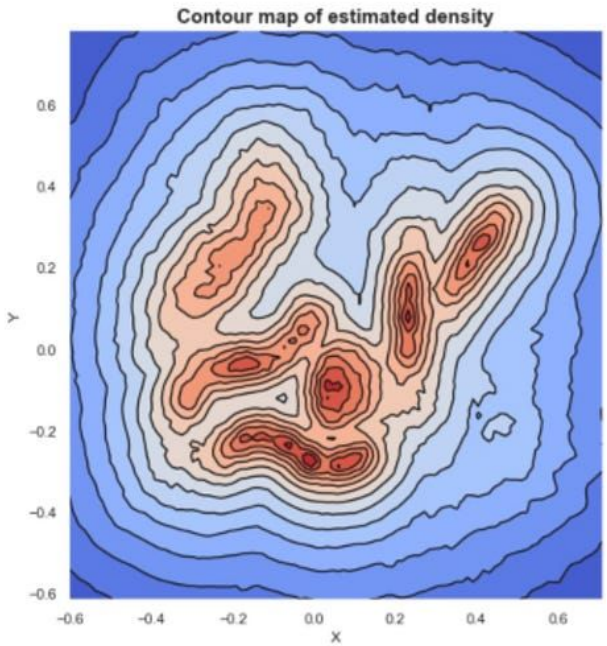
A gentle introduction to HDBSCAN

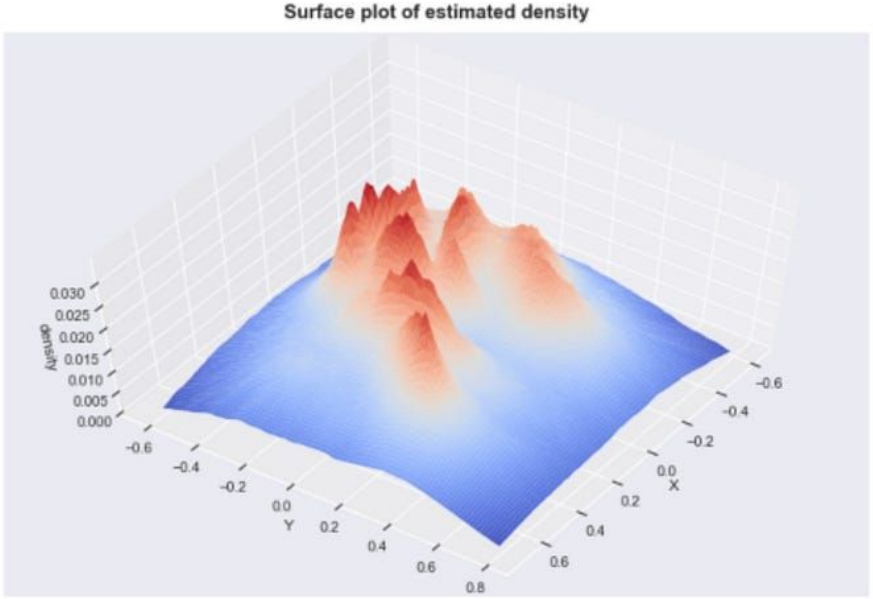
Clustering with HDBSCAN



A gentle introduction to HDBSCAN

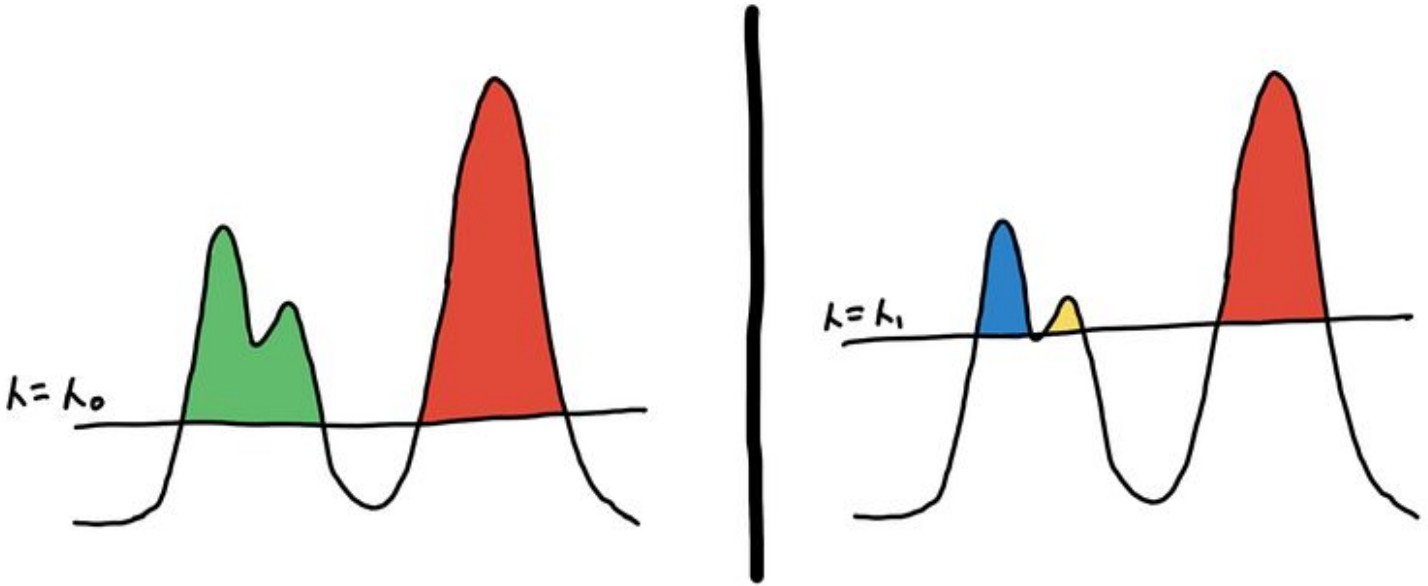
Clustering





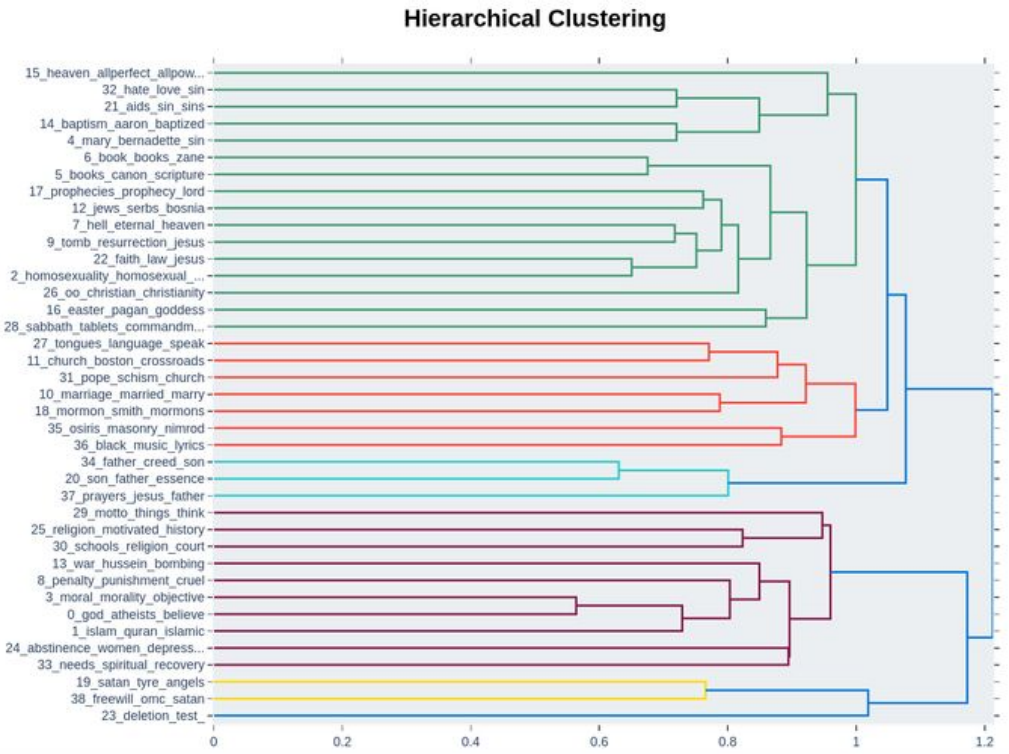
A gentle introduction to HDBSCAN

Clustering with HDBSCAN



A gentle introduction to HDBSCAN

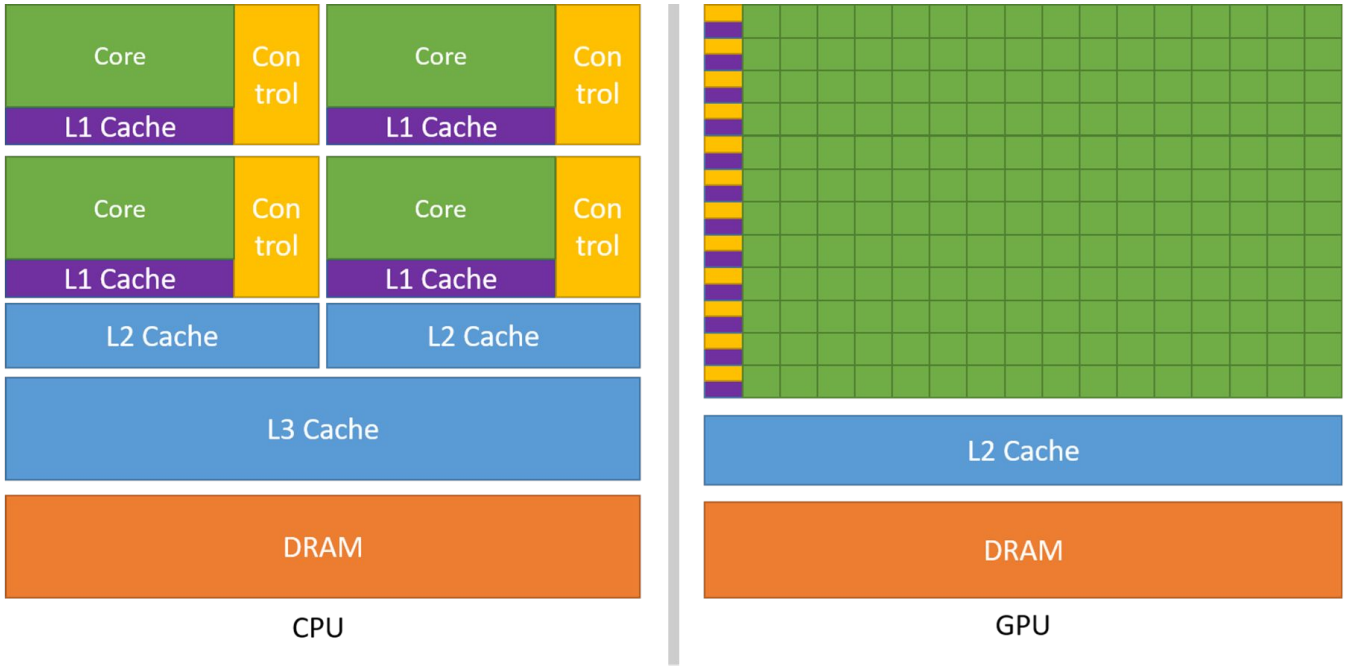
HDBSCAN on 20 Newsgroups dataset



- Dataset

CPU and GPU Comparison

GPUs Leverage Parallel Processing to Handle Large Datasets



https://docs.nvidia.com/cuda/cuda-c-programming-guide/

BERTopic

Use cuML to Speed Up both UMAP and HDBSCAN through GPU Acceleration

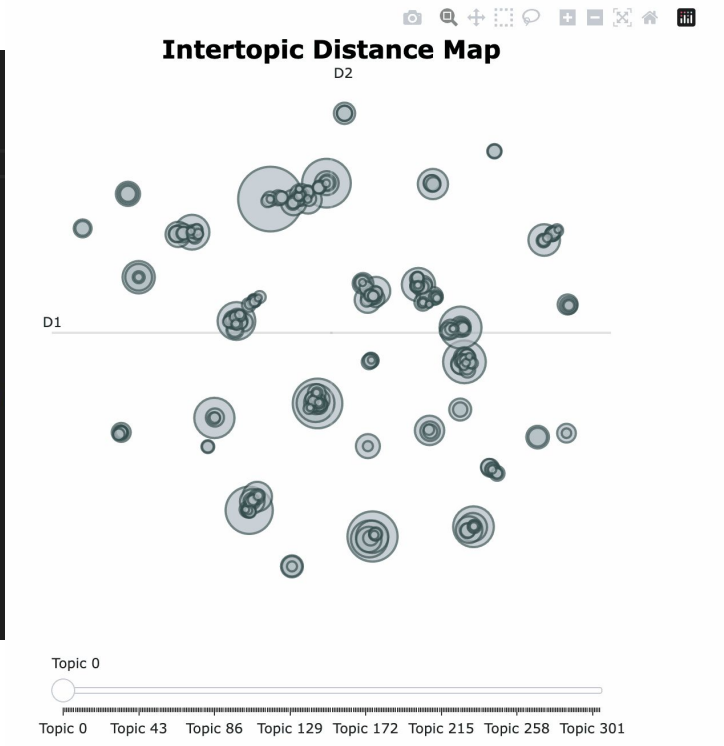
```python
from bertopic import BERTopic
from cuml.cluster import HDBSCAN
from cuml.manifold import UMAP
from datasets import load_dataset

# Load IMDb reviews dataset
dataset = load_dataset("imdb", split="train")
docs = dataset["text"]  # List of text documents

# Create instances of GPU-accelerated UMAP and HDBSCAN
umap_model = UMAP(n_components=5, n_neighbors=15, min_dist=0.0)
hdbscan_model = HDBSCAN(min_samples=10, gen_min_span_tree=True, prediction_data=True)

# Pass the above models to be used in BERTopic
topic_model = BERTopic(umap_model=umap_model, hdbscan_model=hdbscan_model)
topics, probs = topic_model.fit_transform(docs)

# Visualize the topics
topic_model.visualize_topics()
```



Topic keyword extraction with CountVectorizer and c-TF-IDF

Uses original documents in each cluster

Applies simple tokenization (e.g., word-splitting)

Counts word frequencies within clusters

Selects most frequent/representative words

Uses these words to label topics

Review Questions

- How do we create text or image embeddings?
- How do we reduce the dimensionality of embeddings?
- What is the purpose of clustering the embeddings?
- What do we tokenize after we have clustered documents?
- Why do we use c-TF-IDF?

Resources

- Topic Modeling | DGX Station at build.nvidia.com
- Faster Topic Modeling with BERTopic and RAPIDS cuML | by Maarten Grootendorst
- Accelerate Clustering Algorithms to Achieve the Highest Performance | GTC San Jose 2025 | NVIDIA On-Demand
- Practical Computer Vision Bootcamp - GitHub Repository